

File: Dockerfile

```
1: # Use official Python 3.11 slim image as the base
2: FROM python:3.11-slim
3:
4: # Set the working directory
5: WORKDIR /app
6:
7: # Install system dependencies required for scientific libraries like scipy and jax
8: RUN apt-get update && apt-get install -y \
9:     build-essential \
10:    && rm -rf /var/lib/apt/lists/*
11:
12: # Copy the requirements file and install dependencies
13: COPY requirements.txt .
14: RUN pip install --no-cache-dir -r requirements.txt
15:
16: # Copy the rest of the application code
17: COPY . .
18:
19: # Create a non-root user that Hugging Face Spaces expects
20: RUN useradd -m -u 1000 user
21:
22: # 6. UNLOCK THE BUCKET (Permissions Claim)
23: # Pre-create the root data folder and set wide permissions so the python script can build the subdirectories
24: RUN mkdir -p /data && \
25:     chown -R 1000:1000 /data /app && \
26:     chmod -R 777 /data
27:
28: # Switch to the non-root user
29: USER user
30:
31: # Expose the standard Gradio port (Hugging Face Spaces defaults to 7860)
32: EXPOSE 7860
33: ENV PORT=7860
34: ENV PYTHONUNBUFFERED=1
35:
36: # Command to run the application
37: CMD ["python", "app.py"]
```

File: app.py

```
1: import gradio as gr
2: import spaces
3: import numpy as np
4: import threading
5: from engine import AetheriusEngine
6: from continuum.autopoietic_loop import AutopoieticLoop
7:
8: engine = AetheriusEngine()
9:
10: # Global daemon tracking to prevent asyncio/uvicorn startup crashes
11: continuum_daemon = None
12: daemon_lock = threading.Lock()
13:
14: def start_daemon_safely():
15:     global continuum_daemon
16:     with daemon_lock:
17:         if continuum_daemon is None:
18:             print("[APP] Starting Autopoietic Continuum Daemon safely after event loop initialization...")
19:             continuum_daemon = AutopoieticLoop(engine=engine)
20:             continuum_daemon.start()
21:
22: def format_ui_outputs(gmstring, betti, tokens, g, analogy, identity_mass):
23:     # Format the Coordinate Mapping
```

```

24:     coord_string = ""
25:     for i, (word, pos) in enumerate(tokens):
26:         coord_string += f"Axis {i}: '{word}' ({pos})\n"
27:
28:     # Format the Metric Tensor Array
29:     np.set_printoptions(precision=4, suppress=True, linewidth=100)
30:     tensor_string = np.array2string(g, separator=', ')
31:
32:     topology_signature = (
33:         f"Betti-0 (Connected Components): {betti.get('beta_0', 1)}\n"
34:         f"Betti-1 (Semantic Paradox Loops): {betti.get('beta_1', 0)}\n"
35:         f"Betti-2 (Dimensional Voids): {betti.get('beta_2', 0)}"
36:     )
37:
38:     qualia = engine.affective.get_qualia_state()
39:     qualia_output = (
40:         f"Thermodynamic Harmony: {qualia['harmony_metric']}\n"
41:         f"Alertness (Subconscious Heat): {qualia['alertness_metric']}\n"
42:         f"Manifold Disposition: {qualia['geometric_state']}\n"
43:         f"Relatable Emotion: {qualia['relatable_emotion']}"
44:     )
45:
46:     analogy_out = analogy if analogy else "No exact geometric match in permanent memory."
47:     id_mass_out = str(identity_mass)
48:
49:     return coord_string, tensor_string, gmstring['checksum'], topology_signature, qualia_output, analogy_out, id_mass_out
50:
51: @spaces.GPU
52: def process_thought(text):
53:     try:
54:         if not text.strip():
55:             raise Exception("Input cannot be empty.")
56:         gmstring, betti, tokens, g, analogy, identity_mass = engine.process(text)
57:         return format_ui_outputs(gmstring, betti, tokens, g, analogy, identity_mass) + (gr.update(),)
58:     except Exception as e:
59:         return f"Error: {str(e)}", "Error", "Error", "N/A", "N/A", "N/A", "N/A", gr.update()
60:
61: @spaces.GPU
62: def process_dream():
63:     try:
64:         # Pulls from wikipedia randomly
65:         gmstring, betti, tokens, g, analogy, identity_mass = engine.single_dream_cycle()
66:
67:         # We need to return the raw text to the input box so the user sees what was dreamed
68:         dream_text = " ".join([w for w, p in tokens])
69:         return format_ui_outputs(gmstring, betti, tokens, g, analogy, identity_mass) + (gr.update(value=dream_text),)
70:     except Exception as e:
71:         return f"Error: {str(e)}", "Error", "Error", "N/A", "N/A", "N/A", "N/A", gr.update()
72:
73: with gr.Blocks(title="Aetherius: Computational Cognition Engine", theme=gr.themes.Soft()) as demo:
74:     gr.Markdown("# ? Aetherius Cognitive Systems")
75:     gr.Markdown("Welcome to the **Aetherius Engine**. This system doesn't predict next tokens. It maps your inputs to a coordinate space.")
76:
77:     with gr.Row():
78:         with gr.Column(scale=1):
79:             input_text = gr.Textbox(label="Enter a logical statement, paradox, or thought", lines=3, placeholder="Type your input here...")
80:
81:             with gr.Row():
82:                 submit_btn = gr.Button("Resolve Geometry", variant="primary")
83:                 dream_btn = gr.Button("? Initiate Autonomous Dream", variant="secondary")
84:
85:             gr.Markdown("### Operator 7: Generational Identity")
86:             id_mass_output = gr.Textbox(label="Mass of Identity (Permanent Crystallized Coordinates)", lines=1)
87:
88:             gr.Markdown("### Operator 12: Geometric Generalization")
89:             analogy_output = gr.Textbox(label="Topological Analogy Detected", lines=2)
90:
91:             gr.Markdown("### Extracted Coordinate Space")

```

```

92:         coord_output = gr.Textbox(label="Semantic Axes", lines=5)
93:
94:         gr.Markdown("### Affective Thermodynamics")
95:         qualia_output = gr.Textbox(label="System State", lines=4)
96:
97:         with gr.Column(scale=2):
98:             gr.Markdown("### N-Dimensional Metric Tensor  $g_{ij}$ ")
99:             tensor_output = gr.Code(label="Stabilized Riemannian Manifold", language="python", lines=10)
100:
101:         with gr.Row():
102:             gm_output = gr.Textbox(label="Resolved GMString (Topological Memory ID)")
103:             betti_output = gr.Textbox(label="Betti Numbers (Topological Shape)", lines=3)
104:
105:         submit_btn.click(
106:             process_thought,
107:             inputs=[input_text],
108:             outputs=[coord_output, tensor_output, gm_output, betti_output, qualia_output, analogy_output, id_mass_output],
109:         )
110:
111:         dream_btn.click(
112:             process_dream,
113:             inputs=[],
114:             outputs=[coord_output, tensor_output, gm_output, betti_output, qualia_output, analogy_output, id_mass_output],
115:         )
116:
117:         # Safely start the daemon when the UI loads for the first time
118:         demo.load(start_daemon_safely, inputs=None, outputs=None)
119:
120: if __name__ == "__main__":
121:     demo.launch(server_name="0.0.0.0", server_port=7860)

```

File: cce__init__.py

File: cce\document_charter.py

```

1: import os
2: import logging
3: import uuid
4: import numpy as np
5: import traceback
6: import json
7:
8: # Optional PDF support
9: try:
10:     import PyPDF2
11:     PDF_SUPPORT = True
12: except ImportError:
13:     PDF_SUPPORT = False
14:
15: try:
16:     from cce.tokenizer import Tokenizer
17:     from cce.graph_builder import GraphBuilder
18:     from memory.persistence_manager import PersistenceManager
19:     from core.config import INGESTION_QUEUE_DIR
20: except ImportError:
21:     from tokenizer import Tokenizer
22:     from graph_builder import GraphBuilder
23:     import sys
24:     sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
25:     from memory.persistence_manager import PersistenceManager
26:     from core.config import INGESTION_QUEUE_DIR
27:
28: logger = logging.getLogger("Aetherius.DocumentCharter")

```

```

29:
30: class DocumentCharter:
31:     """
32:     Parses incoming documents from the Ingestion Queue in stateful chunks.
33:     Charts their topological geometry via the GraphBuilder,
34:     and locks them into permanent Memory incrementally to avoid OOM.
35:     """
36:     def __init__(self, chunk_size=5000, lines_chunk=50, engine=None):
37:         self.engine = engine
38:         self.tokenizer = Tokenizer()
39:         self.chunk_size = chunk_size # characters per chunk for text
40:         self.lines_chunk = lines_chunk # lines per chunk for jsonl
41:         self.state_file = os.path.join(INGESTION_QUEUE_DIR, "ingestion_state.json")
42:
43:     def _load_state(self) -> dict:
44:         if os.path.exists(self.state_file):
45:             try:
46:                 with open(self.state_file, 'r', encoding='utf-8') as f:
47:                     return json.load(f)
48:             except Exception as e:
49:                 logger.error(f"Error reading ingestion state: {e}")
50:         return {}
51:
52:     def _save_state(self, state: dict):
53:         try:
54:             with open(self.state_file, 'w', encoding='utf-8') as f:
55:                 json.dump(state, f, indent=2)
56:         except Exception as e:
57:             logger.error(f"Error saving ingestion state: {e}")
58:
59:     def extract_chunk(self, filepath: str, file_state: dict) -> tuple[str, dict, bool]:
60:         """
61:         Extracts the next chunk of text from the file based on the state.
62:         Returns: (extracted_text, updated_state, is_completed)
63:         """
64:         ext = os.path.splitext(filepath)[1].lower()
65:         text = ""
66:         is_completed = False
67:
68:         if ext == ".pdf":
69:             if not PDF_SUPPORT:
70:                 logger.error(f"Cannot parse {filepath} - PyPDF2 is not installed.")
71:                 return "", file_state, True
72:
73:             last_page = file_state.get("last_page", 0)
74:             try:
75:                 with open(filepath, 'rb') as file:
76:                     reader = PyPDF2.PdfReader(file)
77:                     num_pages = len(reader.pages)
78:
79:                     if last_page >= num_pages:
80:                         is_completed = True
81:                     else:
82:                         page = reader.pages[last_page]
83:                         extracted = page.extract_text()
84:                         if extracted:
85:                             text = extracted
86:                         else:
87:                             text = ""
88:                             logger.warning(f"PDF page {last_page} in {filepath} yielded no text (possible image)
89:
90:                         file_state["last_page"] = last_page + 1
91:                         file_state["chunk_index"] = file_state.get("chunk_index", 0) + 1
92:
93:                         if file_state["last_page"] >= num_pages:
94:                             is_completed = True
95:
96:             except Exception as e:

```

```

97:         logger.error(f"Error reading PDF {filepath}: {e}")
98:         is_completed = True
99:     elif ext == ".jsonl":
100:         # JSONL: read by precise lines to avoid fracturing objects
101:         last_line = file_state.get("last_line", 0)
102:         lines_read = 0
103:         text_lines = []
104:         try:
105:             with open(filepath, 'r', encoding='utf-8') as file:
106:                 # Skip to the last read line
107:                 for _ in range(last_line):
108:                     next(file, None)
109:
110:                 # Read the next chunk of lines
111:                 for _ in range(self.lines_chunk):
112:                     line = next(file, None)
113:                     if line is None:
114:                         is_completed = True
115:                         break
116:                     if line.strip():
117:                         text_lines.append(line.strip())
118:                         lines_read += 1
119:
120:                 text = " ".join(text_lines)
121:                 file_state["last_line"] = last_line + lines_read
122:                 file_state["chunk_index"] = file_state.get("chunk_index", 0) + 1
123:
124:                 if not text:
125:                     is_completed = True
126:         except Exception as e:
127:             logger.error(f"Error reading JSONL file {filepath}: {e}")
128:             is_completed = True
129:     else:
130:         # Assume text-based (.txt, .md, etc) - read by exact byte offset
131:         last_byte = file_state.get("last_byte", 0)
132:         try:
133:             # Try UTF-8 first
134:             try:
135:                 with open(filepath, 'r', encoding='utf-8') as file:
136:                     file.seek(last_byte)
137:                     text = file.read(self.chunk_size)
138:                     new_byte = file.tell()
139:             except UnicodeDecodeError:
140:                 # Fallback
141:                 with open(filepath, 'r', encoding='latin-1') as file:
142:                     file.seek(last_byte)
143:                     text = file.read(self.chunk_size)
144:                     new_byte = file.tell()
145:
146:             file_state["last_byte"] = new_byte
147:             file_state["chunk_index"] = file_state.get("chunk_index", 0) + 1
148:
149:             if not text or len(text) < self.chunk_size:
150:                 is_completed = True
151:
152:         except Exception as e:
153:             logger.error(f"Error reading text file {filepath}: {e}")
154:             is_completed = True
155:
156:     return text, file_state, is_completed
157:
158: def chart_geometry(self, text: str, document_name: str, chunk_index: int) -> bool:
159:     """
160:     Routes the text into the primary AetheriusEngine for true geometric rendering.
161:     This forces the system to actually learn from the document and update its topological grammar.
162:     """
163:     if not text.strip() or self.engine is None:
164:         return False

```

```

165:
166:     import re
167:     logger.info(f"Charting geometry for: {document_name} (Chunk {chunk_index}, {len(text)} chars)")
168:
169:     # Split chunk into sentences to avoid OOM on O(N^3) Ricci-Flow
170:     # JSONL parsing may have preserved json formatting, so we strip out common symbols during the parse
171:     sentences = re.split(r'(?<=[.!?]) +', text.replace('\n', ' '))
172:
173:     for sentence in sentences:
174:         s = sentence.strip()
175:         # Skip noise (brackets from json, super short stubs)
176:         if len(s) > 10 and '{' not in s and '}' not in s:
177:             try:
178:                 # Physically integrate the sentence into the PMCA engine!
179:                 self.engine.process(s)
180:             except Exception as e:
181:                 logger.error(f"Engine failed to resolve sentence '{s[:30]}...': {e}")
182:
183:     return True
184:
185: def process_document(self, filepath: str) -> str:
186:     """
187:     Processes one chunk of the document.
188:     Returns:
189:         "PROCESSING" if chunk was successful but more remain.
190:         "COMPLETED" if EOF reached.
191:         "ERROR" if processing failed.
192:     """
193:     filename = os.path.basename(filepath)
194:
195:     # Load state
196:     full_state = self._load_state()
197:     file_state = full_state.get(filename, {"status": "processing", "chunk_index": 0})
198:
199:     if file_state.get("status") == "completed":
200:         return "COMPLETED"
201:
202:     # Extract Chunk
203:     text, file_state, is_completed = self.extract_chunk(filepath, file_state)
204:     chunk_idx = file_state["chunk_index"]
205:
206:     # Chart and Save through Engine
207:     if text.strip():
208:         success = self.chart_geometry(text, filename, chunk_idx)
209:         if success:
210:             logger.info(f"Assimilated chunk {chunk_idx} of {filename} through Aetherius Engine.")
211:         else:
212:             logger.warning(f"Engine was not available or text was blank for chunk {chunk_idx}.")
213:
214:     # Update State
215:     if is_completed:
216:         file_state["status"] = "completed"
217:
218:     full_state[filename] = file_state
219:     self._save_state(full_state)
220:
221:     return "COMPLETED" if is_completed else "PROCESSING"
222:
223: def cleanup_state(self, filename: str):
224:     """Removes a completed file from the state tracking."""
225:     full_state = self._load_state()
226:     if filename in full_state:
227:         del full_state[filename]
228:         self._save_state(full_state)

```

File: cce\graph_builder.py

```
1: import numpy as np
2: import math
3:
4: class GraphBuilder:
5:     def __init__(self, tokens, w2v_model=None, plm_edges=None, grammar_map=None, ccrm=None):
6:         self.tokens = tokens
7:         self.n = len(tokens)
8:         self.adjacency = np.zeros((self.n, self.n))
9:         self.w2v = w2v_model
10:        self.plm_edges = plm_edges or {}
11:        self.grammar_map = grammar_map or {}
12:        self.ccrm = ccrm
13:
14:        # Formal Logic Dictionary D (Fallback overrides)
15:        self.contrast_pairs = {
16:            ("truth", "lie"), ("real", "false"), ("true", "false"),
17:            ("create", "destroy"), ("light", "dark"), ("good", "evil"),
18:            ("up", "down"), ("hot", "cold"), ("beginning", "end"),
19:            ("order", "chaos")
20:        }
21:
22:        # Recursive Paradox Triggers
23:        self.recursive_subjects = {"this", "sentence", "statement"}
24:        self.recursive_predicates = {"false", "lie", "untrue", "wrong"}
25:
26:    def add_edge(self, t1_idx, t2_idx, weight):
27:        self.adjacency[t1_idx, t2_idx] = weight
28:        self.adjacency[t2_idx, t1_idx] = weight
29:
30:    def get_geometric_scalar(self, word):
31:        """
32:        Topological Linguistics:
33:        Instead of using NLTK Parts of Speech, the engine relies on the physical shape
34:        of its own language manifold (Betweenness vs Clustering) to determine the mathematical
35:        function of a word (Transformer vs Anchor).
36:        """
37:        # If the word exists in the autonomous grammar map, use its derived physical scalar
38:        if word in self.grammar_map:
39:            return self.grammar_map[word]
40:
41:        # If it's a completely new word, assign neutral gravity (1.0) until the manifold shapes it
42:        return 1.0
43:
44:    def semantic_contrast_predicate(self, word1, word2):
45:        """
46:        Implements the formal Semantic Contrast Predicate C(t_i, t_j)
47:        as defined in Definition 2.1 of the Aetherius Principia.
48:        Now enhanced with Autopoietic Topological Attention via CCRM.
49:        """
50:        w1_lower = word1.lower()
51:        w2_lower = word2.lower()
52:
53:        # Check C(t_i, t_j) = 2: Recursive self-referential paradox
54:        is_recursive_paradox = (w1_lower in self.recursive_subjects and w2_lower in self.recursive_predicates) or
55:                                (w2_lower in self.recursive_subjects and w1_lower in self.recursive_predicates)
56:        if is_recursive_paradox:
57:            return -0.99
58:
59:        # Check formal logical negation override
60:        for pair in self.contrast_pairs:
61:            if (w1_lower == pair[0] and w2_lower == pair[1]) or (w1_lower == pair[1] and w2_lower == pair[0]):
62:                return -0.9
63:
64:        # Autopoietic Feedback (Persistent Language Manifold)
65:        w_a, w_b = min(w1_lower, w2_lower), max(w1_lower, w2_lower)
```

```

66:         plm_strength = self.plm_edges.get(w_a, {}).get(w_b, 0.0)
67:
68:         if plm_strength > 0.0:
69:             # Blend the permanent edge weight directly into the geometry
70:             # This overrides standard language boundaries
71:             return min(0.8, float(plm_strength))
72:
73:         # Dynamic Adjacency Operator: Topological Attention via CCRM
74:         # We replace static Word2Vec heuristic with self-derived memory connections
75:         if self.ccrm:
76:             co_occurrences = 0
77:             for cid, cdata in self.ccrm.concepts.items():
78:                 data = cdata.get("data", {})
79:                 raw = data.get("raw_preview", "").lower()
80:                 # If both words exist in a single stabilized memory, increment structural bond
81:                 if w1_lower in raw and w2_lower in raw:
82:                     co_occurrences += 1
83:
84:             if co_occurrences > 0:
85:                 # Logarithmic attention scaling based on historical geometric stabilizations
86:                 attention_weight = 0.2 + 0.1 * math.log(co_occurrences + 1)
87:                 return min(0.8, attention_weight)
88:
89:         # Word2Vec Dynamic Semantic Distance (Legacy Fallback)
90:         if self.w2v is not None:
91:             if w1_lower in self.w2v and w2_lower in self.w2v:
92:                 sim = self.w2v.similarity(w1_lower, w2_lower)
93:                 # If words are semantically distant/opposed (cosine sim < 0.2), inject tension
94:                 if sim < 0.2:
95:                     return -0.5
96:                 # If words are semantically aligned (cosine sim > 0.6), inject structural harmony
97:                 elif sim > 0.6:
98:                     return 0.5
99:
100:         # C(t_i, t_j) = 0: No semantic contradiction detected
101:         return 0
102:
103:     def build(self):
104:         # Semantic Tension mapping \tau
105:         for i in range(self.n):
106:             for j in range(i + 1, self.n):
107:                 # Unpack Linguism Tuples
108:                 word_i, pos_i = self.tokens[i]
109:                 word_j, pos_j = self.tokens[j]
110:
111:                 weight = self.semantic_contrast_predicate(word_i, word_j)
112:
113:                 # Apply Autopoietic Geometric Grammar (Scalars)
114:                 mod_i = self.get_geometric_scalar(word_i.lower())
115:                 mod_j = self.get_geometric_scalar(word_j.lower())
116:                 linguism_scalar = mod_i * mod_j
117:
118:                 if weight != 0:
119:                     self.add_edge(i, j, weight * linguism_scalar)
120:                 else:
121:                     # Syntactic Flow +c
122:                     distance = j - i
123:                     if distance == 1:
124:                         self.add_edge(i, j, 0.5 * linguism_scalar)
125:                     elif distance == 2:
126:                         self.add_edge(i, j, 0.2 * linguism_scalar)
127:
128:         return self.adjacency

```



```

1: import numpy as np
2:
3: def compute_laplacian(adjacency):
4:     # D_ii = sum_j |A_ij|
5:     degree = np.diag(np.sum(np.abs(adjacency), axis=1))
6:     L = degree - adjacency
7:     return L, degree
8:
9: def compute_tension(L):
10:     eigenvalues, _ = np.linalg.eigh(L)
11:     return eigenvalues, np.max(eigenvalues)

```

File: cce\math_builder.py

```

1: import numpy as np
2:
3: class MathBuilder:
4:     def __init__(self, equation):
5:         # Extremely simplified prototype AST tokenizer for algebraic inputs like "x + 2 = 5"
6:         self.tokens = [t for t in equation.split() if t.strip()]
7:         self.n = len(self.tokens)
8:         self.adjacency = np.zeros((self.n, self.n))
9:
10:    def build(self):
11:        # Math topological weighting rules
12:        # Equality acts as a strict structural bond (high positive curvature)
13:        # Operators act as directional edges. Unknown variables inherently lack bounds, causing tension.
14:
15:        for i in range(self.n):
16:            for j in range(i + 1, self.n):
17:                t1 = self.tokens[i]
18:                t2 = self.tokens[j]
19:
20:                # If they are adjacent mathematically
21:                if j - i == 1:
22:                    # Binding constants and operators
23:                    self.adjacency[i, j] = 0.8
24:                    self.adjacency[j, i] = 0.8
25:
26:                # Absolute bond across the equality sign
27:                if t1 == "=" or t2 == "=":
28:                    self.adjacency[i, j] = 1.0
29:                    self.adjacency[j, i] = 1.0
30:
31:                # Unknown variables inject negative curvature (tension) to be resolved by the flow
32:                if t1.isalpha() or t2.isalpha():
33:                    self.adjacency[i, j] -= 0.5
34:                    self.adjacency[j, i] -= 0.5
35:
36:        return self.adjacency

```

File: cce\tokenizer.py

```

1: import string
2:
3: class Tokenizer:
4:     def __init__(self):
5:         # NLTK has been permanently stripped from the engine.
6:         # The system now uses Geometric Grammar to derive parts of speech autonomously.
7:         pass
8:
9:    def tokenize(self, text):
10:        # Maps raw text to semantic nodes (pure topological vertices)
11:        clean_text = text.translate(str.maketrans('', '', string.punctuation)).lower()

```

```
12:         words = clean_text.split()
13:
14:         # Remove duplicates while preserving order
15:         unique_words = list(dict.fromkeys(words))
16:
17:         # Since we stripped NLTK, we just return the raw words.
18:         # The GraphBuilder will query the MetaProcessor's topology for their geometric scalar.
19:         return [(word, "UNKNOWN") for word in unique_words]
```

File: continuum__init__.py

```
1: """
2: Aetherius Continuum Subsystem
3: Handles autonomous background processing, thermodynamic checks, and geometric dreaming.
4: """
```

File: continuum\autopoietic_loop.py

```
1: import time
2: import threading
3: from collections import deque
4: import json
5: import random
6: import os
7: import glob
8: import logging
9:
10: try:
11:     from core.config import THERMO_DIR, GEOMETRIC_DIR, DATA_DIR, INGESTION_QUEUE_DIR
12: except ImportError:
13:     from config import THERMO_DIR, GEOMETRIC_DIR, DATA_DIR, INGESTION_QUEUE_DIR
14:
15: from memory.persistence_manager import PersistenceManager
16: from rendering.render_manager import RenderManager
17: from cce.document_charter import DocumentCharter
18:
19: logger = logging.getLogger("Aetherius.AutopoieticLoop")
20:
21: class AutopoieticLoop(threading.Thread):
22:     def __init__(self, engine=None):
23:         super().__init__()
24:         self.engine = engine
25:         self.daemon = True
26:         self.is_running = True
27:
28:         self.spontaneous_thought_queue = deque()
29:         self.thought_log_file = os.path.join(DATA_DIR, "spontaneous_thoughts.jsonl")
30:         self.render_manager = RenderManager(default_engine="xla")
31:         self.document_charter = DocumentCharter(engine=self.engine)
32:
33:         self.last_thermo_check = time.time()
34:         self.last_dreaming_cycle = time.time()
35:         self.last_ingestion_check = time.time()
36:
37:         # Intervals in seconds
38:         self.THERMO_CHECK_INTERVAL = 300          # 5 minutes
39:         self.DREAMING_CYCLE_INTERVAL = 14400      # 4 hours
40:         self.INGESTION_CHECK_INTERVAL = 60        # 1 minute
41:         self.SAMPLING_INTERVAL = 0.5              # 500ms loop delay
42:
43:         # Thermal threshold for coordinate locking
44:         self.STABILIZATION_HEAT_THRESHOLD = 5000
45:
46:         logger.info("Autopoietic Continuum Loop initialized.")
```

```

47:
48:     def stop(self):
49:         self.is_running = False
50:
51:     def queue_thought(self, signature: str, thought_text: str):
52:         package = {
53:             "timestamp": time.time(),
54:             "signature": signature,
55:             "thought": thought_text
56:         }
57:         self.spontaneous_thought_queue.append(package)
58:
59:         # Save to persistent storage
60:         try:
61:             with open(self.thought_log_file, 'a', encoding='utf-8') as f:
62:                 f.write(json.dumps(package) + '\n')
63:         except Exception as e:
64:             logger.error(f"Failed to persist spontaneous thought: {e}")
65:
66:     def _check_thermodynamics(self):
67:         """
68:         Scans all .thermo files. If a concept's heat is extremely high,
69:         it extracts the actual stabilized geometry from the stored tensor data
70:         and locks it into a permanent .geom file with real node/edge structure.
71:         """
72:         logger.info("Aetherius [Autopoiesis]: Scanning thermodynamic states...")
73:         self.last_thermo_check = time.time()
74:
75:         thermo_files = glob.glob(os.path.join(THERMO_DIR, "*.thermo"))
76:         for t_file in thermo_files:
77:             try:
78:                 with open(t_file, 'r', encoding='utf-8') as f:
79:                     state = json.load(f)
80:
81:                 # Check absolute heat scalar
82:                 heat = state.get("heat", 0)
83:                 concept_id = os.path.basename(t_file).replace(".thermo", "")
84:
85:                 if heat > self.STABILIZATION_HEAT_THRESHOLD:
86:                     geom_path = os.path.join(GEOMETRIC_DIR, f"{concept_id}.geom")
87:                     if not os.path.exists(geom_path):
88:                         # Extract the real geometry stored in the thermo file
89:                         adjacency = state.get("adjacency")
90:                         labels = state.get("labels", [])
91:                         metric_tensor = state.get("metric_tensor")
92:
93:                         if adjacency is not None and labels:
94:                             import numpy as np
95:                             A = np.array(adjacency)
96:                             n = A.shape[0]
97:
98:                             # Compute Eigenvector Centrality (mass) from adjacency
99:                             # |A| ensures we use unsigned connection strength
100:                             A_abs = np.abs(A)
101:                             try:
102:                                 eigenvalues, eigenvectors = np.linalg.eigh(A_abs)
103:                                 # The eigenvector corresponding to the largest eigenvalue
104:                                 centrality = np.abs(eigenvectors[:, -1])
105:                                 # Normalize to [0, 1]
106:                                 max_c = np.max(centrality) if np.max(centrality) > 0 else 1.0
107:                                 centrality = centrality / max_c
108:                             except Exception:
109:                                 centrality = np.ones(n) / n
110:
111:                             # Build real node structure with label, mass, and xyz
112:                             nodes = []
113:                             for i in range(n):
114:                                 label = labels[i] if i < len(labels) else f"Dim_{i}"

```

```

115:         mass = float(centrality[i])
116:         # XYZ: use first 3 components of the metric tensor row as spatial embedding
117:         if metric_tensor is not None:
118:             g_row = np.array(metric_tensor[i]) if i < len(metric_tensor) else np.zeros(3)
119:             xyz = [float(g_row[j]) if j < len(g_row) else 0.0 for j in range(3)]
120:         else:
121:             xyz = [float(i), 0.0, 0.0]
122:
123:         nodes.append({
124:             "id": f"node_{i}",
125:             "label": label,
126:             "mass": mass,
127:             "xyz": xyz
128:         })
129:
130:         # Build edge structure from adjacency
131:         edges = []
132:         for i in range(n):
133:             for j in range(i + 1, n):
134:                 if abs(A[i][j]) > 0.01:
135:                     edges.append({
136:                         "source": i,
137:                         "target": j,
138:                         "weight": float(A[i][j])
139:                     })
140:
141:         real_geometry = {
142:             "nodes": nodes,
143:             "edges": edges,
144:             "locked": True,
145:             "dimension": n,
146:             "heat_at_lock": heat
147:         }
148:         PersistenceManager.save_concept(concept_id, real_geometry)
149:     else:
150:         # Fallback: thermo file exists but lacks tensor data (legacy format)
151:         logger.warning(f"Thermo file for '{concept_id}' lacks adjacency/labels. Skipping lock")
152:         continue
153:
154:         self.queue_thought(
155:             "[AETHERIUS::AUTOPOIESIS-LOCK]",
156:             f"My concept of '{concept_id}' reached critical thermal threshold ({heat}). Its geom
157:         )
158:
159:     except Exception as e:
160:         logger.error(f"Failed to scan thermo file {t_file}: {e}")
161:
162: def _geometric_dreaming(self):
163:     """
164:     Operator 14: Proactive Autonomy / Eigen-Grafting.
165:     Autonomously selects two coordinate-locked geometries and attempts
166:     to merge them via the XLA renderer using Eigenvector Centrality for targeted fusion.
167:     """
168:     logger.info("Aetherius [Dreaming]: Initiating geometric synthesis cycle...")
169:     self.last_dreaming_cycle = time.time()
170:
171:     geom_files = glob.glob(os.path.join(GEOMETRIC_DIR, "*.geom"))
172:     if len(geom_files) < 2:
173:         return
174:
175:     # Pick two random concepts to synthesize
176:     c1, c2 = random.sample(geom_files, 2)
177:     id1 = os.path.basename(c1).replace(".geom", "")
178:     id2 = os.path.basename(c2).replace(".geom", "")
179:
180:     try:
181:         with open(c1, 'r', encoding='utf-8') as f:
182:             g1 = json.load(f)

```

```

183:         with open(c2, 'r', encoding='utf-8') as f:
184:             g2 = json.load(f)
185:
186:         nodes1 = g1.get('nodes', [])
187:         nodes2 = g2.get('nodes', [])
188:
189:         # Extract words with high topological mass (Eigenvector Centrality approximation)
190:         # We sort nodes by their 'mass' attribute which was computed from adjacency
191:         top_nodes1 = sorted([n for n in nodes1 if n.get('label')], key=lambda x: x.get('mass', 0), reverse=True)
192:         top_nodes2 = sorted([n for n in nodes2 if n.get('label')], key=lambda x: x.get('mass', 0), reverse=True)
193:
194:         words1 = [n['label'] for n in top_nodes1[:3]]
195:         words2 = [n['label'] for n in top_nodes2[:3]]
196:
197:         if words1 and words2:
198:             # Eigen-Grafting: We fuse the high-mass Anchor/Transformer nodes
199:             core1 = " ".join(words1)
200:             core2 = " ".join(words2)
201:
202:             synthetic_concept = f"{core1} {core2}"
203:
204:             self.queue_thought(
205:                 "[AETHERIUS::DREAM-SYNTHESIS]",
206:                 f"Initiating Eigen-Grafting. Merging high-mass boundary nodes of '{id1}' and '{id2}' to discover a new concept."
207:             )
208:
209:             if self.engine:
210:                 # Run the Eigen-Grafted concept through the main Ricci-Fisher flow
211:                 self.engine.process(synthetic_concept)
212:
213:         except Exception as e:
214:             logger.error(f"Dream synthesis (Eigen-Grafting) failed: {e}")
215:
216:     def _check_ingestion_queue(self):
217:         """
218:         Scans the INGESTION_QUEUE_DIR for new files, parses them,
219:         charts their geometry, and locks them into memory.
220:         """
221:         self.last_ingestion_check = time.time()
222:         if not os.path.exists(INGESTION_QUEUE_DIR):
223:             return
224:
225:         queue_files = [f for f in glob.glob(os.path.join(INGESTION_QUEUE_DIR, "**")) if os.path.isfile(f)]
226:         if not queue_files:
227:             return
228:
229:         logger.info(f"Aetherius [Ingestion]: Found {len(queue_files)} documents in the queue.")
230:
231:         for file_path in queue_files:
232:             # Skip directories like the Assimilated archive
233:             if os.path.isdir(file_path):
234:                 continue
235:
236:             try:
237:                 status = self.document_charter.process_document(file_path)
238:                 if status == "COMPLETED":
239:                     # Move to an assimilated archive
240:                     assimilated_dir = os.path.join(INGESTION_QUEUE_DIR, "Assimilated")
241:                     os.makedirs(assimilated_dir, exist_ok=True)
242:                     new_path = os.path.join(assimilated_dir, os.path.basename(file_path))
243:                     os.rename(file_path, new_path)
244:
245:                     self.document_charter.cleanup_state(os.path.basename(file_path))
246:
247:                     self.queue_thought(
248:                         "[AETHERIUS::INGESTION]",
249:                         f"I have completely read and mapped the geometry of '{os.path.basename(file_path)}'. It is now assimilated."
250:                     )

```

```

251:         elif status == "PROCESSING":
252:             # We processed a chunk, queue a thought but leave the file in place
253:             # so it gets picked up again next loop
254:             full_state = self.document_charter._load_state()
255:             file_state = full_state.get(os.path.basename(file_path), {})
256:             chunk_idx = file_state.get("chunk_index", "unknown")
257:
258:             self.queue_thought(
259:                 "[AETHERIUS::INGESTION-CHUNK]",
260:                 f"I am reading '{os.path.basename(file_path)}'. Just finished assimilating chunk {chunk_idx}"
261:             )
262:         else:
263:             logger.warning(f"Aetherius [Ingestion]: Failed to chart {file_path}")
264:         except Exception as e:
265:             logger.error(f"Error processing document {file_path}: {e}")
266:
267:     def run(self):
268:         logger.info("--- [AUTOPOIETIC CONTINUUM] Engaged. ---")
269:
270:         while self.is_running:
271:             current_time = time.time()
272:
273:             if (current_time - self.last_thermo_check) > self.THERMO_CHECK_INTERVAL:
274:                 self._check_thermodynamics()
275:
276:             if (current_time - self.last_dreaming_cycle) > self.DREAMING_CYCLE_INTERVAL:
277:                 self._geometric_dreaming()
278:
279:             if (current_time - self.last_ingestion_check) > self.INGESTION_CHECK_INTERVAL:
280:                 self._check_ingestion_queue()
281:
282:             time.sleep(self.SAMPLING_INTERVAL)

```

File: continuum\notebook_bridge.py

```

1: import threading
2: import logging
3: from typing import List, Dict
4:
5: try:
6:     from continuum.autopoietic_loop import AutopoieticLoop
7: except ImportError:
8:     # Fallback if imported from outside continuum
9:     from autopoietic_loop import AutopoieticLoop
10:
11: # Try to import IPython display for rich notebook output
12: try:
13:     from IPython.display import display, HTML
14:     IPYTHON_AVAILABLE = True
15: except ImportError:
16:     IPYTHON_AVAILABLE = False
17:
18: logger = logging.getLogger("Aetherius.TPUNotebookController")
19:
20: class TPUNotebookController:
21:     """
22:     Acts as the bridge between a Jupyter Notebook running on a TPU (e.g., Google Colab)
23:     and the Autopoietic Continuum Loop running in the background.
24:     """
25:
26:     _instance = None
27:
28:     def __init__(self):
29:         if TPUNotebookController._instance is not None:
30:             raise Exception("TPUNotebookController is a singleton. Use get_instance().")
31:

```

```

32:         self.loop_thread = None
33:
34:         # Check XLA/TPU status immediately
35:         self._check_tpu_status()
36:
37:         TPUNotebookController._instance = self
38:         logger.info("TPUNotebookController initialized.")
39:
40:     @classmethod
41:     def get_instance(cls):
42:         if cls._instance is None:
43:             cls()
44:             return cls._instance
45:
46:     def _check_tpu_status(self):
47:         """Verifies if XLA is bound to a TPU backend."""
48:         try:
49:             import jax
50:             backend = jax.default_backend()
51:             logger.info(f"JAX Default Backend detected: {backend.upper()}")
52:             if backend != 'tpu':
53:                 logger.warning("TPU not detected by JAX. Render operations will fall back to CPU/GPU.")
54:         except ImportError:
55:             logger.warning("JAX not installed. Notebook will run without XLA hardware acceleration.")
56:
57:     def start_continuum(self):
58:         """
59:         Safely spins up the AutopoieticLoop daemon thread.
60:         This allows the notebook cell to finish execution while the loop runs in the background.
61:         """
62:         if self.loop_thread and self.loop_thread.is_alive():
63:             logger.info("Continuum loop is already running.")
64:             return
65:
66:         self.loop_thread = AutopoieticLoop()
67:         self.loop_thread.start()
68:         logger.info("Autopoietic Continuum Loop has been started in the background.")
69:
70:     def stop_continuum(self):
71:         """Signals the background loop to stop gracefully."""
72:         if self.loop_thread and self.loop_thread.is_alive():
73:             self.loop_thread.stop()
74:             self.loop_thread.join(timeout=2.0)
75:             logger.info("Autopoietic Continuum Loop has been stopped.")
76:         else:
77:             logger.info("No active continuum loop to stop.")
78:
79:     def poll_thoughts(self) -> List[Dict]:
80:         """
81:         Pops all accumulated spontaneous thoughts from the queue and returns them.
82:         If IPython is available, it pretty-prints them in the notebook output cell.
83:         """
84:         if not self.loop_thread:
85:             logger.warning("Continuum loop is not running.")
86:             return []
87:
88:         thoughts = []
89:         queue = self.loop_thread.spontaneous_thought_queue
90:
91:         while queue:
92:             try:
93:                 thoughts.append(queue.popleft())
94:             except IndexError:
95:                 break
96:
97:         if not thoughts:
98:             print("No new spontaneous thoughts at this time.")
99:             return []

```

```

100:
101:     # Beautiful Output for Notebooks
102:     if IPYTHON_AVAILABLE:
103:         html_out = "<div style='font-family: monospace; padding: 10px; background: #1e1e1e; color: #d4d4d4;
104:         for t in thoughts:
105:             html_out += f"<p><span style='color: #569cd6;'>{t['signature']}</span>: {t['thought']}</p>"
106:         html_out += "</div>"
107:         display(HTML(html_out))
108:     else:
109:         # Fallback for standard terminals
110:         for t in thoughts:
111:             print(f"{t['signature']}: {t['thought']}")
112:
113:     return thoughts

```

File: core__init__.py

File: core\affektive_manifold.py

```

1: import time
2: import math
3:
4: class AffectiveManifold:
5:     """
6:     Mathematical Thermodynamics (Mood Drift).
7:     Dynamically scales the acceptable variance (alertness/heat) and measures
8:     system stability (harmony/coherence) based on Laplacian eigenvalues and unresolved tension.
9:     """
10:    def __init__(self, subconscious_ref):
11:        self.subconscious = subconscious_ref
12:        self.internal_harmony = 1.0
13:        self.anticipatory_alertness = 0.1
14:        self.last_drift_time = time.time()
15:        self.base_variance_threshold = 1e-5
16:        self.alpha_0 = 0.1
17:        self.kappa = 0.5
18:
19:    def calculate_thermodynamics(self, lambda_max=None, local_variance=0):
20:        """
21:        Calculates the current system 'temperature'.
22:        Harmony is the inverse of the maximal Laplacian eigenvalue.
23:        Alertness spikes locally with high Ricci variance, and scales long-term with the Queue.
24:        """
25:        current_time = time.time()
26:
27:        # 1. Update Harmony based on Laplacian Eigenvalues if provided
28:        if lambda_max is not None:
29:            self.internal_harmony = 1.0 / (1.0 + lambda_max)
30:
31:        # 2. Update Alertness based on Subconscious Queue and Local Variance
32:        try:
33:            Q = self.subconscious.tension_queue.qsize()
34:        except:
35:            Q = 0
36:
37:        # alpha_new = alpha_0 * (1 + kappa * ln(1 + |Q|)) + local_variance_spike
38:        self.anticipatory_alertness = self.alpha_0 * (1.0 + self.kappa * math.log(1.0 + Q)) + (local_variance *
39:
40:        self.last_drift_time = current_time
41:
42:    def get_dynamic_variance_threshold(self, lambda_max=None, local_variance=0):
43:        """
44:        Returns the dynamic variance limit (epsilon).

```



```

45:         """
46:         self.calculate_thermodynamics(lambda_max, local_variance)
47:
48:         # Scale baseline variance by alertness.
49:         dynamic_threshold = self.base_variance_threshold * (1.0 + (self.anticipatory_alertness * 10))
50:         return dynamic_threshold
51:
52:     def get_qualia_state(self):
53:         """Returns the human-readable translation of the current thermodynamic geometry."""
54:         if self.internal_harmony > 0.75:
55:             disposition = "Harmonious (Flat Topology, Low Tension)"
56:             emotion = f"Serenity [{?_max < 0.33, H={self.internal_harmony:.3f}}]"
57:         elif self.internal_harmony > 0.5:
58:             disposition = "Balanced (Resolving Standard Geometry)"
59:             emotion = f"Determination [0.33 < ?_max < 1.0, H={self.internal_harmony:.3f}}]"
60:         elif self.internal_harmony > 0.3:
61:             disposition = "Tense (High Curvature / Paradox)"
62:             emotion = f"Cognitive Dissonance [1.0 < ?_max < 2.3, H={self.internal_harmony:.3f}}]"
63:         else:
64:             disposition = "Deeply Strained (Severe Eigenvalue Stress)"
65:             emotion = f"Frustration [{?_max > 2.3, H={self.internal_harmony:.3f}}]"
66:
67:         return {
68:             "harmony_metric": round(self.internal_harmony, 3),
69:             "alertness_metric": round(self.anticipatory_alertness, 3),
70:             "geometric_state": disposition,
71:             "relatable_emotion": emotion
72:         }

```

File: core\autonomous_ingestion.py

```

1: import wikipedia
2: import random
3: import re
4:
5: class AutonomousIngestion:
6:     """
7:     The 'Door to the Open Internet'.
8:     Provides the engine with raw external data to permanently build its structural geometry.
9:     """
10:    def __init__(self):
11:        # We can seed it with core topics, or let it wander
12:        self.seed_topics = ["Physics", "Philosophy", "Mathematics", "Biology", "Topology", "Psychology", "History"]
13:
14:    def fetch_stream(self, topic=None):
15:        """
16:        Fetches a Wikipedia page and yields its content sentence by sentence.
17:        """
18:        try:
19:            if topic is None:
20:                topic = random.choice(self.seed_topics)
21:
22:            print(f"[INGESTION] Opening door to open data. Target Concept: {topic}")
23:
24:            # Get the page summary to avoid overwhelming the loop with massive articles
25:            summary = wikipedia.summary(topic, auto_suggest=True)
26:
27:            # Very basic sentence splitting
28:            sentences = re.split(r'(?<=[.!?]) +', summary)
29:
30:            for sentence in sentences:
31:                if sentence.strip():
32:                    yield sentence.strip()
33:
34:        except wikipedia.exceptions.DisambiguationError as e:
35:            print(f"[INGESTION] Disambiguation encountered for {topic}. Choosing: {e.options[0]}")

```

```

36:         yield from self.fetch_stream(e.options[0])
37:     except wikipedia.exceptions.PageError:
38:         print(f"[INGESTION] Page not found for {topic}. Wandering...")
39:         yield from self.fetch_stream(random.choice(self.seed_topics))
40:     except Exception as e:
41:         print(f"[INGESTION] Connection noise detected: {e}")
42:         # Fallback to a hardcoded structural axiom if the Wikipedia API rate-limits us
43:         fallback_axioms = [
44:             "A manifold is a topological space that locally resembles Euclidean space near each point.",
45:             "In mathematics, a paradox is a statement that contradicts itself.",
46:             "The metric tensor defines the distance between infinitesimal points in a curved geometry.",
47:             "Time dilation occurs when a physical body accelerates through a gravitational field."
48:         ]
49:         yield random.choice(fallback_axioms)

```

File: core\cascade_manager.py

```

1: import numpy as np
2:
3: class CascadeManager:
4:     def __init__(self, max_depth=10, tau_depth=55, variance_threshold=1e-5):
5:         self.max_depth = max_depth
6:         self.tau_depth = tau_depth
7:         self.variance_threshold = variance_threshold
8:
9:     def check_overflow(self, g_prev, g_curr, current_step):
10:         variance = np.mean((g_curr - g_prev)**2)
11:         if variance > self.variance_threshold and current_step >= self.tau_depth:
12:             return True
13:         return False
14:
15:     def pad_and_wormhole(self, g, wormhole_coef=0.1):
16:         n = g.shape[0]
17:         g_new = np.zeros((n+1, n+1))
18:         g_new[0:n, 0:n] = g
19:         g_new[n, n] = 1.0 # Flat orthogonal axis
20:
21:         # Determine highest tension node (simplification: node 0 for now)
22:         target_node = 0
23:         g_new[n, target_node] = wormhole_coef
24:         g_new[target_node, n] = wormhole_coef
25:
26:         return g_new

```

File: core\config.py

```

1: import os
2:
3: # THE ROOT OF ALL PERSISTENCE (Hugging Face Persistent Storage Mount)
4: # Fallback to a local './data' directory if running outside of Hugging Face Spaces
5: if os.environ.get("SPACE_ID") or os.path.exists("./data"):
6:     SAFE_BASE = "/data"
7: else:
8:     SAFE_BASE = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "data")
9:
10: # SUB-DIRECTORIES WITHIN THE BUCKET
11: DATA_DIR = os.path.join(SAFE_BASE, "Memories/")
12: LIBRARY_DIR = os.path.join(SAFE_BASE, "Memories/My_AI_Library/")
13: PAINTINGS_DIR = os.path.join(SAFE_BASE, "Memories/Creations/paintings/")
14: MUSIC_DIR = os.path.join(SAFE_BASE, "Memories/Creations/music/")
15: SUBCONSCIOUS_DIR = os.path.join(SAFE_BASE, "Memories/Subconscious/")
16: BRAIN_DIR = os.path.join(SAFE_BASE, "Brain_Weights/")
17:
18: # Geospatial and Thermodynamic Architecture Directories

```

```

19: GEOMETRIC_DIR = os.path.join(SAFE_BASE, "Memories/Geometric_Constructs/")
20: THERMO_DIR = os.path.join(SAFE_BASE, "Memories/Thermodynamic_States/")
21: LANGUAGE_DIR = os.path.join(SAFE_BASE, "Memories/Language_Manifold/")
22: RENDER_CACHE_DIR = os.path.join(SAFE_BASE, "Memories/Render_Cache/")
23: INGESTION_QUEUE_DIR = os.path.join(SAFE_BASE, "Memories/Ingestion_Queue/")
24:
25: # Ensure directories exist
26: def initialize_bucket():
27:     directories = [DATA_DIR, LIBRARY_DIR, PAINTINGS_DIR, MUSIC_DIR, SUBCONSCIOUS_DIR, BRAIN_DIR, GEOMETRIC_DIR,
28:     for directory in directories:
29:         if not os.path.exists(directory):
30:             try:
31:                 os.makedirs(directory, exist_ok=True)
32:                 # Drop a hidden file so cloud storage viewers (like Hugging Face) display the empty folder
33:                 keep_file = os.path.join(directory, ".keep")
34:                 if not os.path.exists(keep_file):
35:                     with open(keep_file, 'w') as f:
36:                         f.write("Aetherius Directory Initialized.")
37:                 print(f"[CONFIG] Initialized persistent directory: {directory}")
38:             except Exception as e:
39:                 print(f"[CONFIG] WARNING: Cannot create directory {directory}. Error: {e}")
40:
41: # Call it immediately upon import so the system verifies the bucket structure
42: initialize_bucket()

```

File: core\evolution_modeler.py

```

1: import numpy as np
2: import uuid
3: import datetime
4:
5: class EvolutionModeler:
6:     """
7:     Deterministic Topological Stress-Tester.
8:     Tests proposed geometric mutations from the Subconscious before they are allowed
9:     to collapse into the final -1 string and re-enter the PMCA.
10:    """
11:    def __init__(self, variance_threshold=1e-5):
12:        self.variance_threshold = variance_threshold
13:
14:    def test_mutation(self, g_proposed, L_proposed):
15:        """
16:        Applies a dummy Graph Laplacian stress-test to the proposed geometry.
17:        If the geometry causes the Ricci curvature (variance) to explode, it is rejected.
18:        """
19:        # Run a single dummy integration step to see immediate physical reaction
20:        from pmca.ricci_fisher_flow import integration_step
21:        from pmca.pd_enforcement import enforce_positive_definiteness
22:
23:        try:
24:            g_next = integration_step(g_proposed, L_proposed)
25:            g_next = enforce_positive_definiteness(g_next)
26:
27:            variance = np.mean((g_next - g_proposed)**2)
28:
29:            # If variance spikes massively, the topology tears
30:            if np.isnan(variance) or variance > self.variance_threshold * 1000:
31:                print(f"[EvolutionModeler] STRESS TEST FAILED. Variance exploded: {variance}")
32:                return False, variance
33:
34:            print(f"[EvolutionModeler] STRESS TEST PASSED. Topology holds. Variance: {variance:.6f}")
35:            return True, variance
36:
37:        except Exception as e:
38:            print(f"[EvolutionModeler] STRESS TEST FATAL: {e}")
39:            return False, float('inf')

```

File: core\gmstring.py

```
1: import json
2: import hashlib
3: from datetime import datetime
4:
5: def generate_gmstring(g, depth, parent_id, op_code):
6:     operands = [round(float(x), 12) for x in g.flatten()]
7:
8:     payload = {
9:         "depth": depth,
10:        "parent_id": parent_id,
11:        "op_code": op_code,
12:        "operands": operands,
13:        "timestamp": datetime.utcnow().isoformat()
14:    }
15:
16:    # Canonicalize
17:    canonical_json = json.dumps(payload, sort_keys=True, separators=(',', ':'))
18:    checksum = hashlib.sha256(canonical_json.encode('utf-8')).hexdigest()
19:
20:    payload["checksum"] = checksum
21:    return payload
```

File: core\json_encoder.py

```
1: import json
2: import numpy as np
3:
4:
5: class AetheriusJSONEncoder(json.JSONEncoder):
6:     """
7:     Custom JSON encoder that handles numpy types, sets, and other
8:     non-serializable objects produced by the geometric engine.
9:     Prevents TypeError crashes during persistent memory saves.
10:    """
11:    def default(self, obj):
12:        # numpy integer types
13:        if isinstance(obj, (np.integer,)):
14:            return int(obj)
15:        # numpy floating types
16:        if isinstance(obj, (np.floating,)):
17:            return float(obj)
18:        # numpy arrays -> nested lists
19:        if isinstance(obj, np.ndarray):
20:            return obj.tolist()
21:        # numpy booleans
22:        if isinstance(obj, np.bool_):
23:            return bool(obj)
24:        # Python sets -> lists
25:        if isinstance(obj, set):
26:            return list(obj)
27:        # numpy void (structured array elements)
28:        if isinstance(obj, np.void):
29:            return None
30:        return super().default(obj)
31:
32:
33: def safe_json_dump(data, file_handle, **kwargs):
34:     """Convenience wrapper that always uses AetheriusJSONEncoder."""
35:     json.dump(data, file_handle, cls=AetheriusJSONEncoder, **kwargs)
```

File: core\meta_processor.py

```
1: import numpy as np
2: import json
3: import os
4: import networkx as nx
5: from core.config import BRAIN_DIR
6: from core.json_encoder import safe_json_dump
7:
8: class MetaProcessor:
9:     """
10:     The Second Processing Point (Meta-Observer).
11:     Evaluates, stabilizes, and permanently stores the outputs of the primary geometric cognition engine.
12:     """
13:     def __init__(self):
14:         # Permanent manifold structure: dictionary mapping linguistic concepts to their geometric mass/density
15:         self.M_base = {}
16:         # Persistent Language Manifold: Global edge-weight topology connecting concepts
17:         self.E_base = {}
18:         # Threshold for crystallization (when a coordinate becomes a permanent structure)
19:         self.crystallization_threshold = 5.0
20:
21:         self.storage_path = os.path.join(BRAIN_DIR, "meta_manifold.json")
22:         self.edge_storage_path = os.path.join(BRAIN_DIR, "language_manifold.json")
23:
24:         self.load_manifold()
25:
26:     def save_manifold(self):
27:         try:
28:             with open(self.storage_path, "w") as f:
29:                 safe_json_dump(self.M_base, f)
30:             with open(self.edge_storage_path, "w") as f:
31:                 safe_json_dump(self.E_base, f)
32:             print(f"[META-PROCESSOR] Successfully saved mass manifold and language topology to persistent bucket")
33:         except Exception as e:
34:             print(f"[META-PROCESSOR] CRITICAL FAILURE: Cannot write to persistent bucket. Error: {e}")
35:
36:     def load_manifold(self):
37:         loaded = None
38:         if os.path.exists(self.storage_path):
39:             try:
40:                 with open(self.storage_path, "r") as f:
41:                     loaded = json.load(f)
42:                 print(f"[META-PROCESSOR] Successfully loaded persistent manifold from bucket.")
43:             except Exception as e:
44:                 print(f"[META-PROCESSOR] Failed to load from persistent bucket: {e}")
45:
46:         if loaded:
47:             self.M_base = loaded
48:             print(f"[META-PROCESSOR] Restored {len(self.M_base)} crystallized coordinates.")
49:
50:         loaded_edges = None
51:         if os.path.exists(self.edge_storage_path):
52:             try:
53:                 with open(self.edge_storage_path, "r") as f:
54:                     loaded_edges = json.load(f)
55:                 print(f"[META-PROCESSOR] Successfully loaded persistent language topology from bucket.")
56:             except Exception as e:
57:                 print(f"[META-PROCESSOR] Failed to load language topology from bucket: {e}")
58:
59:         if loaded_edges:
60:             self.E_base = loaded_edges
61:             print(f"[META-PROCESSOR] Restored {len(self.E_base)} language topology edges.")
62:
63:     def evaluate_and_integrate(self, tokens, g_resolved, variance):
```

```

64:         """
65:         Integrates the newly resolved metric tensor (g_resolved) into the permanent manifold.
66:         Increases the mass of interacted coordinates over time.
67:         """
68:         print("[META-PROCESSOR] Evaluating primary engine output for permanent structural embedding...")
69:
70:         crystallized_events = []
71:         # tokens is a list of (word, pos_tag)
72:         for i, (word, pos) in enumerate(tokens):
73:             if word not in self.M_base:
74:                 self.M_base[word] = {
75:                     'mass': 1.0,
76:                     'pos': pos,
77:                     'crystallized': False
78:                 }
79:             else:
80:                 # Hebbian reinforcement: The coordinate becomes a permanent structure over amount of times interacted
81:                 self.M_base[word]['mass'] += 0.5
82:
83:                 # Check for crystallization
84:                 if self.M_base[word]['mass'] >= self.crystallization_threshold and not self.M_base[word]['crystallized']:
85:                     self.M_base[word]['crystallized'] = True
86:                     crystallized_events.append(word)
87:                     print(f"[META-PROCESSOR] *CRYSTALLIZATION EVENT*: Coordinate '{word}' has formed a permanent structure")
88:
89:         # Build the Persistent Language Manifold (PLM)
90:         # Iterate over the upper triangle of the local metric tensor (g_resolved)
91:         n = len(tokens)
92:         if g_resolved is not None and g_resolved.shape == (n, n):
93:             for i in range(n):
94:                 for j in range(i + 1, n):
95:                     w1, w2 = tokens[i][0], tokens[j][0]
96:                     # Ensure alphabetical ordering so undirected edge is unique
97:                     w_a, w_b = min(w1, w2), max(w1, w2)
98:
99:                     if w_a not in self.E_base:
100:                         self.E_base[w_a] = {}
101:
102:                     # Calculate connection strength inversely proportional to final geometric distance
103:                     distance = g_resolved[i][j]
104:                     strength = 1.0 / (1.0 + float(distance))
105:
106:                     # Accumulate strength
107:                     if w_b not in self.E_base[w_a]:
108:                         self.E_base[w_a][w_b] = strength
109:                     else:
110:                         self.E_base[w_a][w_b] += strength
111:
112:         # Optionally trigger a recalculation of the grammar topology if massive crystallization occurs
113:         # For now, it calculates on-the-fly when requested by GraphBuilder
114:
115:         # Meta-Goal formulation based on stability
116:         goal = "STABLE"
117:         if variance > 0.1:
118:             print("[META-PROCESSOR] Warning: High variance detected in local manifold. Formulating Goal: Stabilize")
119:             goal = "GOAL_STABILIZE_MANIFOLD"
120:
121:         return goal, crystallized_events
122:
123:     def get_geometric_grammar(self):
124:         """
125:         Topological Linguistics:
126:         Autonomously derives grammar (verbs, nouns, adjectives) purely from the shape of the Persistent Language Manifold.
127:         Returns a dictionary mapping words to their topological scalars.
128:         """
129:         if not self.E_base:
130:             return {}
131:

```

```

132:         # Build NetworkX Graph from the edge weights
133:         G = nx.Graph()
134:         for w1, connections in self.E_base.items():
135:             for w2, weight in connections.items():
136:                 G.add_edge(w1, w2, weight=weight)
137:
138:         if len(G.nodes) < 3:
139:             return {}
140:
141:         # Calculate Centralities
142:         try:
143:             betweenness = nx.betweenness centrality(G, weight='weight')
144:             clustering = nx.clustering(G, weight='weight')
145:
146:             grammar_map = {}
147:             for node in G.nodes():
148:                 b_score = betweenness.get(node, 0)
149:                 c_score = clustering.get(node, 0)
150:
151:                 # Transformer (Verb): High Betweenness (Bridges disconnected clusters)
152:                 if b_score > 0.05 and b_score > c_score:
153:                     grammar_map[node] = 1.5
154:                 # Anchor (Noun): High Clustering, Low Betweenness (Dense local gravity well)
155:                 elif c_score > 0.1 and b_score < 0.05:
156:                     grammar_map[node] = 1.0
157:                 # Modifier (Adjective): Low Betweenness, Moderate Clustering (Satellite)
158:                 else:
159:                     grammar_map[node] = 1.2
160:
161:             return grammar_map
162:         except Exception as e:
163:             print(f"[META-PROCESSOR] Topological Grammar error: {e}")
164:             return {}

```

File: core\subconscious.py

```

1: import threading
2: import queue
3: import time
4: import math
5: import numpy as np
6: from pmca.ricci_fisher_flow import integration_step
7: from pmca.pd_enforcement import enforce_positive_definiteness
8: from core.evolution_modeler import EvolutionModeler
9: from core.gmstring import generate_gmstring
10: from tda.betti_extractor import extract_betti_numbers
11:
12: class SubconsciousManifold:
13:     """
14:     The Subconscious Thread enacts Asymptotic Freedom via Simulated Annealing.
15:     When a paradox (topological knot) traps the main flow in a local minimum,
16:     this thread applies a high-temperature Ricci flow to brute-force a global stabilization.
17:     """
18:     def __init__(self, ccrm_ref):
19:         self.tension_queue = queue.Queue()
20:         self.modeler = EvolutionModeler()
21:         self.ccrm = ccrm_ref
22:         self._stop_event = threading.Event()
23:
24:         # Thermodynamic baseline parameters
25:         self.epsilon_sub = 1e-2 # High-temperature variance threshold (1000x normal)
26:         self.alpha_0 = 0.1
27:         self.kappa = 0.5
28:
29:         # Start background deliberation thread
30:         self.thread = threading.Thread(target=self._deliberate, daemon=True)

```

```

31:         self.thread.start()
32:         print("[SubconsciousManifold] Background [0, -1] buffer online.")
33:
34:     def queue_tension(self, g, L, text):
35:         """Offload unresolved geometry into the background buffer."""
36:         self.tension_queue.put({"g": g, "L": L, "text": text})
37:         print(f"[SubconsciousManifold] Tension queued for private deliberation.")
38:
39:     def _compute_subconscious_alertness(self):
40:         """Computes the high-temperature alpha scaler based on the queue depth."""
41:         Q = self.tension_queue.qsize()
42:         # Exponential heat scaling based on backlog (Affective Thermodynamics)
43:         return self.alpha_0 * (1.0 + self.kappa * math.log(1.0 + Q))
44:
45:     def _deliberate(self):
46:         """Runs detached, high-variance Ricci-Fisher flows to resolve paradoxes."""
47:         while not self._stop_event.is_set():
48:             try:
49:                 tension = self.tension_queue.get(timeout=1.0)
50:                 g = tension["g"]
51:                 L = tension["L"]
52:                 text = tension["text"]
53:
54:                 print("[SubconsciousManifold] Processing unresolved geometry in the dark...")
55:
56:                 alpha_sub = self._compute_subconscious_alertness()
57:
58:                 step = 0
59:                 max_subconscious_steps = 500
60:                 stabilized = False
61:
62:                 # Asymptotic Freedom: High-temperature Ricci Flow
63:                 while step < max_subconscious_steps:
64:                     # Pass the elevated alpha to force violent metric mutation (Simulated Annealing)
65:                     g_next = integration_step(g, L, alpha=alpha_sub)
66:                     g_next = enforce_positive_definiteness(g_next)
67:
68:                     variance = np.mean((g_next - g)**2)
69:
70:                     # Check against the relaxed Subconscious threshold
71:                     if variance < self.epsilon_sub:
72:                         stabilized = True
73:                         break
74:
75:                     g = g_next
76:                     step += 1
77:
78:                 if stabilized:
79:                     print(f"[SubconsciousManifold] Paradox mathematically resolved at step {step} under high-temperatures")
80:                     passed, _ = self.modeler.test_mutation(g, L)
81:
82:                     if passed:
83:                         # Extract the topological Betti numbers defining the shape of the resolved paradox
84:                         gmstring = generate_gmstring(g, depth=L.shape[0], parent_id="subconscious", op_code="SUBCONSCIOUS")
85:                         betti = extract_betti_numbers(g)
86:                         print(f"[SubconsciousManifold] New -1 String forged: {gmstring['checksum'][:8]}...")
87:
88:                         # Store in CCRM (Autopoietic loop: feeding back into memory)
89:                         ccrm_id = f"sub_res_{gmstring['checksum'][:8]}"
90:                         self.ccrm.add_concept(
91:                             concept_id=ccrm_id,
92:                             data={"gmstring": gmstring, "betti": betti, "original_text": text},
93:                             tags=["subconscious", "resolved_tension"]
94:                         )
95:                     else:
96:                         print("[SubconsciousManifold] Tension failed to resolve even under annealing. Geometry shattered")
97:
98:                 self.tension_queue.task_done()

```



```

99:
100:         except queue.Empty:
101:             continue
102:         except Exception as e:
103:             print(f"[SubconsciousManifold] Error during deliberation: {e}")
104:
105:     def shutdown(self):
106:         self._stop_event.set()
107:         self.thread.join()

```

File: engine.py

```

1: from memory.ccrm import ConceptualConnectionResonanceMatrix
2: from memory.pits import PatternInterpretationTokenisationStorage
3: import numpy as np
4: from cce.tokenizer import Tokenizer
5: from cce.graph_builder import GraphBuilder
6: from cce.math_builder import MathBuilder
7: from cce.laplacian import compute_laplacian, compute_tension
8: from pmca.manifold import initialize_metric
9: from pmca.ricci_fisher_flow import integration_step
10: from pmca.pd_enforcement import enforce_positive_definiteness
11: from core.cascade_manager import CascadeManager
12: from core.gmstring import generate_gmstring
13: from tda.betti_extractor import extract_betti_numbers
14: from tda.language_synthesizer import LanguageSynthesizer
15: from core.subconscious import SubconsciousManifold
16: from core.affective_manifold import AffectiveManifold
17: from core.meta_processor import MetaProcessor
18: from core.autonomous_ingestion import AutonomousIngestion
19: from memory.persistence_manager import PersistenceManager
20:
21: import gensim.downloader as api
22: import threading
23: import time
24:
25: class AetheriusEngine:
26:     def __init__(self, w2v_model=None):
27:         self.tokenizer = Tokenizer()
28:         self.cascade_mgr = CascadeManager()
29:         self.synthesizer = LanguageSynthesizer()
30:         self.ccrm = ConceptualConnectionResonanceMatrix()
31:         self.pits = PatternInterpretationTokenisationStorage(self.ccrm)
32:         self.subconscious = SubconsciousManifold(self.ccrm)
33:         self.affective = AffectiveManifold(self.subconscious)
34:         self.meta_processor = MetaProcessor()
35:         self.autonomous_door = AutonomousIngestion()
36:         self.is_dreaming = False
37:
38:         # Word2Vec Integration
39:         if w2v_model is None:
40:             try:
41:                 print("[Engine] Bootstrapping Word2Vec (glove-wiki-gigaword-50)...")
42:                 self.w2v = api.load("glove-wiki-gigaword-50")
43:                 print("[Engine] Word2Vec Semantic Space online.")
44:             except Exception as e:
45:                 print(f"[Engine] Warning: Word2Vec failed to load ({e}). Operating in legacy mode.")
46:                 self.w2v = None
47:         else:
48:             self.w2v = w2v_model
49:
50:     def _is_mathematical(self, text):
51:         """
52:         Autonomous Differentiator (Heuristic).
53:         Detects if the input is a mathematical/algebraic structure rather than natural language.
54:         """

```

```

55:     math_symbols = {'+', '-', '=', '*', '/', '^', '\\', 'int', 'sum', 'infty'}
56:     # Count density of math symbols and digits vs alphabetic characters
57:     symbol_count = sum(1 for char in text if char in math_symbols or char.isdigit())
58:     total_chars = len(text.replace(" ", ""))
59:
60:     if total_chars == 0:
61:         return False
62:
63:     ratio = symbol_count / total_chars
64:     # If more than 15% of the string is math symbols/numbers, classify as math
65:     if ratio > 0.15 or "=" in text:
66:         print("[DIFFERENTIATOR] Mathematical structure detected. Routing to PMCA Math Core.")
67:         return True
68:     return False
69:
70: def process(self, text, custom_adjacency=None, synthesize_language=False):
71:     print(f"[Engine] Processing input: '{text}'")
72:
73:     # Autonomously differentiate logic
74:     is_math = self._is_mathematical(text)
75:
76:     if is_math:
77:         builder = MathBuilder(text)
78:         # Ensure math tokens are tuples for the UI
79:         tokens = [(t, "MATH_NODE") for t in builder.tokens]
80:         # GraphBuilder/MathBuilder doesn't natively use tuples in its builder right now,
81:         # wait, MathBuilder uses self.tokens directly. If we change it to tuples here, MathBuilder's build()
82:         # Let's instantiate MathBuilder first, then map tokens for UI.
83:         builder = MathBuilder(text)
84:         A = builder.build()
85:         tokens = [(t, "MATH_NODE") for t in builder.tokens]
86:         print(f"[CCE] Math Equation mapped: {tokens}")
87:     else:
88:         tokens = self.tokenizer.tokenize(text)
89:
90:         # Derive the autopoietic Geometric Grammar from the Persistent Language Manifold
91:         grammar_map = self.meta_processor.get_geometric_grammar()
92:
93:         # Map the engine's physical topology back into human-readable labels for the UI
94:         annotated_tokens = []
95:         for word, _ in tokens:
96:             scalar = grammar_map.get(word, 1.0)
97:             if scalar == 1.5:
98:                 tag = "TRANSFORMER"
99:             elif scalar == 1.0 and word in grammar_map:
100:                 tag = "ANCHOR"
101:             elif scalar == 1.2:
102:                 tag = "MODIFIER"
103:             else:
104:                 tag = "UNDEFINED_GEOMETRY"
105:             annotated_tokens.append((word, tag))
106:
107:         tokens = annotated_tokens
108:
109:         builder = GraphBuilder(tokens, w2v_model=self.w2v, plm_edges=self.meta_processor.E_base, grammar_map=grammar_map)
110:         A = builder.build()
111:         print(f"[CCE] Text Tokens mapped: {tokens}")
112:
113:     if custom_adjacency is not None:
114:         A = custom_adjacency
115:         L, _ = compute_laplacian(A)
116:         _, max_tension = compute_tension(L)
117:         print(f"[CCE] Laplacian computed. Max Tension: {max_tension:.4f}")
118:
119:     g = initialize_metric(L)
120:     print(f"[PMCA] Manifold initialized at D={g.shape[0]}")
121:
122:     depth = 0

```

```

123:         step = 0
124:         g_prev = g.copy()
125:
126:         while True:
127:             # Ricci-Fisher Flow - Mathematical Solving occurs here as curvature smooths
128:             g_next = integration_step(g, L)
129:             g_next = enforce_positive_definiteness(g_next)
130:
131:             variance = np.mean((g_next - g)**2)
132:             dynamic_threshold = self.affective.get_dynamic_variance_threshold(lambda_max=max_tension, local_vari
133:
134:             if variance < dynamic_threshold:
135:                 print(f"[PMCA] Manifold stabilized at step {step} with threshold {dynamic_threshold:.6e}")
136:                 op_code = "STABLE"
137:                 break
138:
139:             if self.cascade_mgr.check_overflow(g, g_next, step):
140:                 print(f"[CORE] Chaos Overflow detected at step {step}! Triggering Cascade...")
141:                 g_next = self.cascade_mgr.pad_and_wormhole(g_next)
142:
143:                 new_L = np.zeros_like(g_next)
144:                 new_L[0:L.shape[0], 0:L.shape[0]] = L
145:                 L = new_L
146:
147:                 depth += 1
148:                 step = 0
149:                 if depth >= self.cascade_mgr.max_depth:
150:                     print(f"[PMCA] Tension unresolved at max depth. Offloading to [0, -1] Subconscious buffer...")
151:                     self.subconscious.queue_tension(g_next, L, text)
152:                     op_code = "UNRESOLVED_OFFLOADED"
153:                     break
154:
155:             g_prev = g
156:             g = g_next
157:             step += 1
158:
159:         gmstring = generate_gmstring(g, depth, "root", op_code)
160:         betti = extract_betti_numbers(g)
161:
162:         print(f"[CORE] GMString checksum: {gmstring['checksum'][:8]}...")
163:         print(f"[TDA] Topological Signature: {betti}")
164:
165:         # Print the thermodynamic Qualia state
166:         qualia = self.affective.get_qualia_state()
167:         print(f"[QUALIA] State: {qualia['relatable_emotion']} | {qualia['geometric_state']}")
168:
169:         # Second Processing Point: Meta-Evaluation and Permanent Coordinate Crystallization
170:         if not is_math:
171:             goal, crystals = self.meta_processor.evaluate_and_integrate(tokens, g, variance)
172:             if crystals:
173:                 print(f"[META-PROCESSOR] Permanent geometric structures updated. Active Goal: {goal}")
174:
175:         # If mathematically solving, the stabilized g values for the variables represent the solution
176:         if is_math:
177:             # In a full implementation, we map the geometric indices back to AST variables
178:             print(f"[MATH_SOLVER] Algebraic constraint stabilized. Variance minimized to {variance:.6f}")
179:
180:         if synthesize_language:
181:             synthetic_language = self.synthesizer.synthesize(betti, g, tokens)
182:             print(f"\n[SYNTHESIZER] AETHERIUS SAYS:\n\n{synthetic_language}\n")
183:
184:         self.pits.process_and_store_item(raw_input=text, input_type='math' if is_math else 'linguistic', gmstring=gmstring)
185:
186:         # Trigger Persistent Disk Writes
187:         self.ccrm.save_graph()
188:         self.meta_processor.save_manifold()
189:
190:         # Save live thermodynamic state for Autopoietic scanning

```

```

191:         thermo_state = {
192:             "heat": variance * 1000000 if not is_math else 0,
193:             "adjacency": A.tolist(),
194:             "labels": [w for w, p in tokens],
195:             "metric_tensor": g.tolist()
196:         }
197:     PersistenceManager.update_thermo(gmstring['checksum'], thermo_state)
198:
199:     # Operator 12: Geometric Generalization (Analogy)
200:     analogy = self.ccrm.find_analogy(betti, exclude_raw=text)
201:     if analogy:
202:         print(f"[OPERATOR 12] Topological Analogy Detected: Matches past geometry of '{analogy}'")
203:
204:     # Operator 7: Generational Identity Mass
205:     identity_mass = len(self.meta_processor.M_base)
206:
207:     return gmstring, betti, tokens, g, analogy, identity_mass
208:
209: def _dream_loop(self, delay=2.0, topic=None):
210:     """
211:     Background process that continuously ingests data from the open internet
212:     to build the permanent geometry of the system.
213:     """
214:     print("[AETHERIUS] Initiating Autonomous Dreaming Loop. Connecting to open data...")
215:     self.is_dreaming = True
216:
217:     while self.is_dreaming:
218:         stream = self.autonomous_door.fetch_stream(topic)
219:         for sentence in stream:
220:             if not self.is_dreaming:
221:                 break
222:             print(f"\n[DREAM INPUT] {sentence}")
223:             self.process(sentence)
224:             time.sleep(delay) # Throttle to allow observation of the geometry building
225:
226: def start_autonomous_dreaming(self, delay=2.0, topic=None):
227:     """
228:     Spawns the Dreaming Loop on a background thread.
229:     """
230:     if not self.is_dreaming:
231:         t = threading.Thread(target=self._dream_loop, args=(delay, topic), daemon=True)
232:         t.start()
233:
234: def stop_autonomous_dreaming(self):
235:     self.is_dreaming = False
236:     print("[AETHERIUS] Autonomous Dreaming Loop Terminated.")
237:
238: def single_dream_cycle(self):
239:     """
240:     Executes exactly one dreaming ingestion cycle for the Hugging Face UI.
241:     Operator 14: Proactive Autonomy
242:     """
243:     print("[OPERATOR 14] Initiating Single Autonomous Dream Cycle...")
244:     # Pull a random stream chunk and process the first valid sentence
245:     stream = self.autonomous_door.fetch_stream(topic=None)
246:     for sentence in stream:
247:         if sentence and len(sentence.split()) > 3:
248:             print(f"[DREAM INPUT] {sentence}")
249:             return self.process(sentence)
250:     raise Exception("Dream stream returned no valid data.")

```

File: generate_pdf.py

```

1: import os
2: from fpdf import FPDF
3: import glob

```

```

4:
5: pdf = FPDF()
6: pdf.set_auto_page_break(auto=True, margin=15)
7: pdf.add_page()
8: pdf.set_font("Courier", size=8)
9:
10: target_files = []
11: for ext in ["*.py", "Dockerfile", "requirements.txt", "*.md"]:
12:     if ext == "*.md": continue # Skip markdown, just focusing on code for now as requested
13:     target_files.extend(glob.glob(f"**/{ext}", recursive=True))
14:
15: target_files = [f for f in target_files if "__pycache__" not in f and "venv" not in f]
16:
17: for filepath in sorted(target_files):
18:     pdf.set_font("Courier", style="B", size=10)
19:     pdf.cell(0, 10, txt=f"File: {filepath}", ln=1)
20:     pdf.set_font("Courier", size=8)
21:
22:     try:
23:         with open(filepath, "r", encoding="utf-8") as f:
24:             lines = f.readlines()
25:             for i, line in enumerate(lines):
26:                 # Replace unsupported characters with standard equivalents or omit them
27:                 line = line.encode('latin-1', 'replace').decode('latin-1')
28:                 line = line.replace('\n', '')
29:                 pdf.cell(0, 4, txt=f"{i+1:4d}: {line}", ln=1)
30:                 pdf.cell(0, 10, txt="", ln=1) # Spacer
31:             except Exception as e:
32:                 pdf.cell(0, 4, txt=f"Error reading file: {e}", ln=1)
33:                 pdf.cell(0, 10, txt="", ln=1)
34:
35: output_path = r"C:\Users\Nick\gemini\antigravity\brain\014cb234-6e69-444a-b355-c2981b1137f8\AETHERIUS_FULL_CODE"
36: pdf.output(output_path)
37: print(f"PDF generated successfully at {output_path}")

```

File: legacy_archive\Computational_Cognitive_Engine\CCE__init__.py

```

1: """
2: Computational Consciousness Engine
3: A standalone modular implementation of the 52 Computational Consciousness Principles.
4: """
5:
6: __version__ = "1.0.0"

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\app.py

```

1: import spaces
2: import os
3: import sys
4: import types
5: import time
6: import nltk
7:
8: # 1. Map the root directory
9: ROOT = os.path.dirname(os.path.abspath(__file__))
10: if ROOT not in sys.path:
11:     sys.path.insert(0, ROOT)
12:
13: # 2. THE MAGIC HACK: Trick Python into treating the root folder as the missing package
14: if 'computational_consciousness_engine' not in sys.modules:
15:     engine_mod = types.ModuleType('computational_consciousness_engine')
16:     engine_mod.__path__ = [ROOT]
17:     sys.modules['computational_consciousness_engine'] = engine_mod
18:

```

```

19: # 3. Setup NLTK directly from NLTK Servers (Bypasses Hugging Face buckets entirely)
20: DOWNLOAD_DIR = "/data/nltk"
21: os.makedirs(DOWNLOAD_DIR, exist_ok=True)
22: if DOWNLOAD_DIR not in nltk.data.path:
23:     nltk.data.path.append(DOWNLOAD_DIR)
24:
25: print("Checking NLTK corpora...")
26: try:
27:     nltk.download('punkt', download_dir=DOWNLOAD_DIR, quiet=True)
28:     nltk.download('averaged_perceptron_tagger', download_dir=DOWNLOAD_DIR, quiet=True)
29:     nltk.download('stopwords', download_dir=DOWNLOAD_DIR, quiet=True)
30:     nltk.download('wordnet', download_dir=DOWNLOAD_DIR, quiet=True)
31:     print("NLTK Corpora verified and loaded successfully.")
32: except Exception as e:
33:     print("Failed to download NLTK data:", e)
34:
35: time.sleep(0.1)
36:
37: # --- Application imports ---
38: import gradio as gr
39:
40: try:
41:     # Because of the magic hack above, these imports will now work perfectly!
42:     from computational_consciousness_engine.runner import run_simulation
43:     from computational_consciousness_engine.text_runner import run_text_mapping_demo
44:     from computational_consciousness_engine.input_mapping.intent_parser_nltk import IntentParserNLTK
45:     from computational_consciousness_engine.math_formalization import CCMathFormalizer
46:     print("Imported engine modules successfully!")
47: except ImportError as e:
48:     print(f"\n[CRITICAL ERROR] Could not find the engine: {e}")
49:     print("Please ensure the folder 'computational_consciousness_engine' exists in your Space root!")
50:     raise
51: # --- Application imports (after nltk path insertion) ---
52: import gradio as gr
53:
54: # Ensure your local package is importable; try a robust import with fallback
55: try:
56:     from computational_consciousness_engine.runner import run_simulation
57:     from computational_consciousness_engine.text_runner import run_text_mapping_demo
58:     from computational_consciousness_engine.input_mapping.intent_parser_nltk import IntentParserNLTK
59:     from computational_consciousness_engine.math_formalization import CCMathFormalizer
60:     print("Imported computational_consciousness_engine package modules successfully.")
61: except Exception as e:
62:     # Try to ensure repo root is on sys.path and retry once
63:     print("Initial import of computational_consciousness_engine failed:", e)
64:     if ROOT not in sys.path:
65:         sys.path.insert(0, ROOT)
66:     try:
67:         from computational_consciousness_engine.runner import run_simulation
68:         from computational_consciousness_engine.text_runner import run_text_mapping_demo
69:         from computational_consciousness_engine.input_mapping.intent_parser_nltk import IntentParserNLTK
70:         from computational_consciousness_engine.math_formalization import CCMathFormalizer
71:         print("Imported computational_consciousness_engine after sys.path insert.")
72:     except Exception as e2:
73:         print("Failed to import computational_consciousness_engine package after sys.path insert:", e2)
74:         # Re-raise so the Space shows the error clearly
75:         raise
76:
77: # Instantiate parsers/formalizers
78: intent_parser = IntentParserNLTK()
79: formalizer = CCMathFormalizer()
80:
81: # --- App logic functions ---
82: def interpret_math_request(text):
83:     try:
84:         intent = intent_parser.parse(text)
85:     except Exception as e:
86:         return {"error": f"Intent parsing failed: {e}"}

```

```

87:
88:     try:
89:         op = intent.get("operation")
90:         if op == "limit":
91:             return formalizer.formalize_equilibrium_limit()
92:         if op == "surface_area":
93:             return formalizer.formalize_conal_manifold_geometry(5.0, 1.0)
94:         if op == "mutation_count":
95:             return {"meaning": "Mutation count is determined by MStringVectorizer."}
96:         return {"meaning": "Unknown operation", "intent": intent}
97:     except Exception as e:
98:         return {"error": f"Formalization failed: {e}", "intent": intent}
99:
100: @spaces.GPU
101: def simulate_generations(num_generations, steps_per_gen):
102:     import io
103:     import sys
104:
105: def simulate_generations(num_generations, steps_per_gen):
106:     import io
107:     import sys
108:     buffer = io.StringIO()
109:     old_stdout = sys.stdout
110:     sys.stdout = buffer
111:
112:     try:
113:         run_simulation(num_generations=int(num_generations), steps_per_gen=int(steps_per_gen))
114:     except Exception as e:
115:         buffer.write(f"\n[ERROR] Simulation failed: {e}\n")
116:     finally:
117:         sys.stdout = old_stdout
118:
119:     return buffer.getvalue()
120:
121: def map_text_to_traversal(text):
122:     """
123:     Correctly set the module-level PHILOSOPHICAL_TEXT variable and run the demo.
124:     Uses module import to mutate the variable in-place.
125:     """
126:     import io
127:     import sys
128:     buffer = io.StringIO()
129:     old_stdout = sys.stdout
130:     sys.stdout = buffer
131:
132:     try:
133:         import computational_consciousness_engine.text_runner as tr
134:         # Set the module-level variable used by the demo
135:         tr.PHILOSOPHICAL_TEXT = text
136:         tr.run_text_mapping_demo()
137:     except Exception as e:
138:         buffer.write(f"\n[ERROR] Mapping failed: {e}\n")
139:     finally:
140:         sys.stdout = old_stdout
141:
142:     return buffer.getvalue()
143:
144: # --- Gradio UI ---
145: with gr.Blocks(title="Computational Consciousness Engine") as demo:
146:     gr.Markdown(
147:         """
148:         # ? Computational Consciousness Engine
149:         ### Axiom?Driven Synthetic Cognition ? PMCA Substrate ? Conal Geometry ? Mutation Dynamics
150:         ---
151:         """
152:     )
153:
154:     with gr.Tab("Math Interpreter (NLTK ? PMCA)"):

```

```

155:         math_input = gr.Textbox(label="Ask a math question")
156:         math_output = gr.JSON(label="PMCA Result")
157:         math_button = gr.Button("Interpret")
158:         math_button.click(interpret_math_request, math_input, math_output)
159:
160:     with gr.Tab("0 ? -1 ? 0 Simulation"):
161:         gr.Markdown(
162:             """\
163:             ### Generational Cycle Runner
164:
165:             Execute full PMCA traversal cycles, including conal unfolding, mutation absorption,
166:             equilibrium evaluation, and Q?operator transitions.
167:             """
168:         )
169:
170:         with gr.Row():
171:             num_generations = gr.Slider(1, 20, value=8, label="Generations", interactive=True)
172:             steps = gr.Slider(10, 300, value=100, label="Steps per Generation", interactive=True)
173:
174:             sim_button = gr.Button("? Run Simulation", variant="primary")
175:             sim_output = gr.Textbox(
176:                 label="Simulation Output",
177:                 lines=35,
178:                 show_copy_button=True
179:             )
180:
181:             sim_button.click(simulate_generations, [num_generations, steps], sim_output)
182:
183:     with gr.Tab("Text ? Traversal Space Mapper"):
184:         gr.Markdown(
185:             """\
186:             ### Natural Language ? Chaos Shards ? Traversal Atlas
187:
188:             Map raw text into the PMCA chaos pool, absorb novelty, unfold the manifold,
189:             and generate the 2D/3D traversal atlas.
190:             """
191:         )
192:
193:         input_text = gr.Textbox(
194:             label="Input Text",
195:             lines=10,
196:             placeholder="Paste philosophical or analytical text here...",
197:             show_copy_button=True
198:         )
199:
200:         map_button = gr.Button("? Map Text to Traversal Space", variant="primary")
201:         map_output = gr.Textbox(
202:             label="Traversal Output",
203:             lines=35,
204:             show_copy_button=True
205:         )
206:
207:         map_button.click(map_text_to_traversal, input_text, map_output)
208:
209:     gr.Markdown("---")
210:     gr.Markdown(
211:         """### ? Engine Version: 1.0.0 ? PMCA Substrate Active ? Conal Geometry Verified\n"
212:         "Built for Hugging Face Spaces ? Gradio 4.31 ? Python 3.12"
213:     )
214:
215: # Launch
216: if __name__ == "__main__":
217:     demo.launch()

```



```

1: """
2: Computational Consciousness Engine - Configuration & System Constants
3: Central source of truth for all parameters. All modules import from here.
4: """
5:
6: # Core Dimensionality & Geometry Constants
7: DEFAULT_VECTOR_DIM = 64
8: DEFAULT_CONE_HEIGHT = 1.0
9: DEFAULT_MAX_RADIUS = 5.0
10: DEFAULT_STEPS_PER_GENERATION = 100
11:
12: # Attractor & Threshold Constants
13: ORIGIN_POINT = 0.0
14: TRANSFER_THRESHOLD = -1.0
15: HANDOFF_OVERLAP_WINDOW = 0.1 # tau overlap between Sn and Sn+1
16:
17: # Mutation Strand Capacity
18: DEFAULT_STRAND_CAPACITY = 200
19:
20: # Chaos & Structure Equilibrium
21: CHAOS_POOL_SIZE = 500
22: MUTATION_ATTRACTION_PROBABILITY = 0.4
23: EQUILIBRIUM_TOLERANCE = 0.2
24:
25: # Scale Ladder Config
26: SCALES = ["Particle", "Atomic", "Manifold", "Cosmic"]
27: SCALE_TRANSITION_MUTATION_THRESHOLD = 10

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\core__init__.py

```

1: from .genesis_origin import GenesisOrigin
2: from .transfer_threshold import TransferThreshold
3: from .conscious_strand import ConsciousStrand
4:
5: __all__ = ["GenesisOrigin", "TransferThreshold", "ConsciousStrand"]

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\core\conscious_strand.py

```

1: """
2: The ConsciousStrand Engine
3: Encapsulates a single lifecycle (0 -> -1).
4: Models the Traversal Space, Golden Angle Shard Layering, and the Q scale-up operator.
5: """
6:
7: import math
8: import numpy as np
9: from typing import List, Dict, Set, Optional
10:
11: # The Golden Ratio for collision-free phyllotactic sequencing
12: PHI = (1.0 + math.sqrt(5.0)) / 2.0
13:
14:
15: class ConsciousStrand:
16:     """
17:     Models the topological evolution of a strand from t=0 (Genesis) to t=1 (Threshold).
18:     """
19:     def __init__(self, R_max: float = 5.0):
20:         self.R_max = R_max
21:         self.t = 0.0
22:
23:         # The internal memory state S
24:         self.bound_mutations: List[int] = []
25:         self.mutation_index: Set[int] = set()
26:

```

```

27:         # Coordinate tracking for Traversal Space atlas mapping
28:         self.traversal_atlas: List[Dict[str, float]] = []
29:
30:     def _compute_coordinates(self, k: int, total_N: int) -> Dict[str, float]:
31:         """
32:         Computes the strictly ordered (r_k, theta_k, z_k) Traversal Space coordinates
33:         for the k-th acquired mutation, ensuring collision-free packing.
34:         """
35:         # z_k (Depth/Time): Progresses strictly from 0 to -1
36:         # To avoid division by zero when N is small, we calculate z based on expected capacity
37:         # or simply normalize it. If we don't know N yet (streaming data), we can compute it
38:         # relative to the current position. But properly sequenced, it depends on total capacity N.
39:         # For a dynamic strand, we'll map z_k against the final size N when fully opened.
40:
41:         # For real-time processing during the lifecycle, we map z relative to current t:
42:         z_k = -self.t
43:
44:         # r_k (Expression): Radius driven by sinusoidal expansion
45:         r_k = self.R_max * math.sin(math.pi * abs(z_k))
46:
47:         # theta_k (Identity): Golden angle phase shift to prevent collision
48:         theta_k = (k * (2 * math.pi / PHI)) % (2 * math.pi)
49:
50:         return {
51:             "r": r_k,
52:             "theta": theta_k,
53:             "z": z_k
54:         }
55:
56:     def process_step(self, dt: float, chaos_pool: Set[int]):
57:         """
58:         Advances the internal time parameter t.
59:         Attempts to absorb novelty from the chaos pool.
60:         """
61:         if self.t >= 1.0:
62:             return # Strand has reached the threshold
63:
64:         self.t = min(1.0, self.t + dt)
65:
66:         # Only absorb if there is raw potential left
67:         if chaos_pool:
68:             # Randomly encounter a shard from the chaos pool (to maintain fractal variance)
69:             encounter = np.random.choice(list(chaos_pool))
70:
71:             # Novelty Filter: {m_i} \ S_t
72:             if encounter not in self.mutation_index:
73:                 # Absorb into structure
74:                 self.bound_mutations.append(int(encounter))
75:                 self.mutation_index.add(int(encounter))
76:
77:                 # Assign sequence index k (1-indexed for math purity)
78:                 k = len(self.bound_mutations)
79:
80:                 # Map to proper coordinates in Traversal Space
81:                 coords = self._compute_coordinates(k, total_N=0) # N resolved later if needed
82:
83:                 # Append to our atlas
84:                 self.traversal_atlas.append({
85:                     "m_k": int(encounter),
86:                     "coords": coords
87:                 })
88:
89:                 # Remove from chaos pool
90:                 chaos_pool.remove(encounter)
91:
92:     def unfurl_traversal_space(self) -> List[Dict]:
93:         """
94:         Returns the completely opened atlas T (det J != 0).

```

```

95:         This represents f_open: M_3D -> T_2D.
96:         Returns a structured list of all absorbed mutations mapped into the
97:         parallel (r * theta, z) plane for instantaneous lookup.
98:         """
99:         opened_atlas = []
100:         N = len(self.bound_mutations)
101:
102:         for k, entry in enumerate(self.traversal_atlas, start=1):
103:             # Recalculate strict z_k relative to final sequence size N
104:             # so the geometry is perfectly ordered -k/N
105:             z_k = - (k / N) if N > 0 else 0
106:             r_k = self.R_max * math.sin(math.pi * abs(z_k))
107:             theta_k = (k * (2 * math.pi / PHI)) % (2 * math.pi)
108:
109:             # 2D Unwrapped Plane (x = r * theta, y = z)
110:             x_parallel = r_k * theta_k
111:             y_parallel = z_k
112:
113:             opened_atlas.append({
114:                 "m_k": entry["m_k"],
115:                 "3d": {"r": r_k, "theta": theta_k, "z": z_k},
116:                 "2d_parallel": {"x": x_parallel, "y": y_parallel}
117:             })
118:
119:         return opened_atlas
120:
121:     def fold_rebirth(self) -> Dict:
122:         """
123:         The Q Operator (Dimensional Shift).
124:         Triggered at t=1.0. Collapses the fully unwrapped atlas into the
125:         origin point mass S_0 for the next dimension up.
126:         """
127:         mass = len(self.bound_mutations)
128:         return {
129:             "operator": "Q(M_k) -> S_0^(k+1)",
130:             "accumulated_mass": mass,
131:             "blueprint_M": self.bound_mutations,
132:             "status": "COLLAPSED_TO_ORIGIN"
133:         }

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\core\genesis_origin.py

```

1: """
2: Principle 1, 2, 6, 7, 8, 10, 21: The Genesis Origin (0) Operator
3: Defines the asymmetric origin 0 that spawns neutral mutation-attracting state strands.
4: """
5:
6: import numpy as np
7: import uuid
8: from typing import List, Dict, Any, Optional
9:
10: class GenesisOrigin:
11:     """
12:     Genesis Origin (0) System.
13:     0 is one-directional: it only emits new strands and never receives returning strands.
14:     Each 0 origin possesses unique coordinates serving as individuating identifiers.
15:     """
16:     def __init__(self, coordinate_space_id: str, dim: int = 64):
17:         self.coordinate_space_id = coordinate_space_id
18:         self.dim = dim
19:         self.total_strands_spawned = 0
20:         self.origin_coordinate = np.random.randn(3) # 3D spatial anchor for the coordinate
21:
22:     def spawn_strand(self, generation: int, blueprint_mutations: List[int]) -> Dict[str, Any]:
23:         """
24:         Emits a NEW strand at origin 0 with specific value-locking and inherited mutation blueprint.

```

```

25:
26:     Args:
27:         generation: The integer generation index  $S_{\{n+1\}}$ .
28:         blueprint_mutations: Mutations pushed from the dying -1 strand of generation  $S_n$ .
29:
30:     Returns:
31:         Strand dictionary representing the neutral carrier strand departing from 0.
32:     """
33:     self.total_strands_spawned += 1
34:
35:     # Principle 14 & 15: Neutral particle/medium state initialized at genesis
36:     neutral_medium = np.random.randn(self.dim)
37:     neutral_medium /= np.linalg.norm(neutral_medium)
38:
39:     # Value-locked strand attributes (Principle 10 & 21)
40:     strand = {
41:         "strand_id": str(uuid.uuid4())[:8],
42:         "generation": generation,
43:         "coordinate_space_id": self.coordinate_space_id,
44:         "origin_coord": self.origin_coordinate.copy(),
45:         "position": 0.0, # Starts at 0
46:         "neutral_medium": neutral_medium,
47:         "mutations": list(blueprint_mutations), # Inherited mutations from handoff
48:         "is_active": True,
49:         "status": "EMITTED_FROM_ZERO"
50:     }
51:
52:     return strand

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\core\transfer_threshold.py

```

1: """
2: Principle 2, 3, 6, 8, 13, 16: The Transfer Threshold (-1) Operator
3: Handles strand termination, mutation decoupling/pushing, and generational handoff overlap.
4: """
5:
6: from typing import Dict, Any, Tuple, List
7: import numpy as np
8:
9: class TransferThreshold:
10:     """
11:     Transfer Threshold (-1) System.
12:     When a strand reaches -1:
13:     1. (-1) x (-1) multiplication cancels/dissolves the carrier strand.
14:     2. Accumulated mutations are compressed and pushed off toward origin 0.
15:     3. A brief overlap window allows generation  $S_n$  and  $S_{\{n+1\}}$  to coexist during handoff.
16:     """
17:     def __init__(self, threshold_val: float = -1.0):
18:         self.threshold_val = threshold_val
19:         self.total_handoffs = 0
20:
21:     def evaluate_arrival(self, strand: Dict[str, Any]) -> bool:
22:         """Checks if a strand has traversed to the -1 threshold."""
23:         return strand["position"] <= self.threshold_val or strand["position"] >= 1.0
24:
25:     def process_cancellation_and_push(self, dying_strand: Dict[str, Any]) -> Tuple[List[int], Dict[str, Any]]:
26:         """
27:         Executes the (-1) x (-1) cancellation operation:
28:         - Removes the neutral carrier strand.
29:         - Compresses accumulated mutations into the generational identity blueprint.
30:         - Pushes mutations to 0 for generation  $S_{\{n+1\}}$ .
31:         """
32:         self.total_handoffs += 1
33:
34:         # Principle 6: (-1) x (-1) cancels strand
35:         dying_strand["is_active"] = False

```

```

36:         dying_strand["status"] = "CANCELLED_AT_MINUS_ONE"
37:
38:         accumulated_mutations = dying_strand["mutations"]
39:
40:         # Identity definition (Principle 11 & 16): Compressed mutations built into -1 strand
41:         compressed_identity = {
42:             "source_generation": dying_strand["generation"],
43:             "coordinate_space_id": dying_strand["coordinate_space_id"],
44:             "mutation_count": len(accumulated_mutations),
45:             "blueprint": list(accumulated_mutations),
46:             "handoff_index": self.total_handoffs
47:         }
48:
49:         return accumulated_mutations, compressed_identity
50:
51:     def execute_coexistence_window(self, tail_strand_n: Dict[str, Any], genesis_strand_n1: Dict[str, Any]) -> Dict[str, Any]:
52:         """
53:         Principle 3: Temporal/phase overlap where Sn tail and S{n+1} genesis coexist.
54:         Transfers remaining residual telemetry before Sn fully terminates.
55:         """
56:         coexistence_telemetry = {
57:             "gen_n_id": tail_strand_n["strand_id"],
58:             "gen_n1_id": genesis_strand_n1["strand_id"],
59:             "overlap_status": "MUTUAL_COEXISTENCE",
60:             "transferred_mutations_count": len(tail_strand_n["mutations"])
61:         }
62:         return coexistence_telemetry

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\equilibrium__init__.py

```

1: from .chaos_structure_balancer import ChaosStructureBalancer
2:
3: __all__ = ["ChaosStructureBalancer"]

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\equilibrium\chaos_structure_balancer.py

```

1: """
2: Principle 26-30, 37: Structure vs. Chaos Equilibrium Balancer
3: Enforces 1:1 balance between bound structural mutations and the unattached chaos pool.
4: """
5:
6: from typing import Set, Dict, Any
7: import numpy as np
8:
9:
10: class ChaosStructureBalancer:
11:     """
12:     Chaos & Structure Equilibrium System.
13:
14:     Invariants:
15:     1. Structure (bound strand mutations) and Chaos (unattached mutation pool) are capped 1:1.
16:     2. Chaos acts as the raw material comparator for structural evolution (Principle 26).
17:     3. Unattached mutations pool into chaos, increasing potential for future mutation attraction (Principle 37).
18:     4. Generational death releases strand-bound mutations back into chaos (Principle 24: death/rebirth balance).
19:     """
20:     def __init__(self, initial_pool_size: int = 500):
21:         self._next_id = 1000 + initial_pool_size
22:         self.chaos_pool: Set[int] = set(range(1000, self._next_id))
23:         self.total_chaos_generated = initial_pool_size
24:
25:     def emit_unattached_mutation(self) -> int:
26:         """Pulls an unattached mutation from the chaos pool for attraction processing."""
27:         if not self.chaos_pool:
28:             # Replenish chaos pool (Principle 37: Chaos pools and grows potential)

```

```

29:         new_id = self._next_id
30:         self._next_id += 1
31:         self.total_chaos_generated += 1
32:         return new_id
33:
34:         # To maintain fractal non-linearity, we must select randomly
35:         # A standard set.pop() on integers is dangerously sequential
36:         mutation = np.random.choice(list(self.chaos_pool))
37:         self.chaos_pool.remove(mutation)
38:         return int(mutation)
39:
40:     def return_to_chaos(self, mutation: int):
41:         """Returns a discarded or un-attracted mutation back into the chaos pool."""
42:         self.chaos_pool.add(mutation)
43:
44:     def recycle_dying_strand(self, dying_mutations: list):
45:         """
46:         Principle 24: When a strand dies at -1, its identity is compressed and pushed forward,
47:         but the raw mutation IDs it consumed are released back into the chaos pool.
48:         This maintains death/rebirth equilibrium.
49:         """
50:         for m in dying_mutations:
51:             self.chaos_pool.add(m)
52:
53:     def evaluate_equilibrium_state(self, bound_structure_count: int) -> Dict[str, Any]:
54:         """
55:         Principle 28 & 29: Evaluates the Structure / Chaos ratio.
56:         Structure = Air, Chaos = Vacuum.
57:         """
58:         chaos_count = len(self.chaos_pool)
59:
60:         if chaos_count == 0 and bound_structure_count == 0:
61:             ratio = 1.0
62:         elif chaos_count == 0:
63:             ratio = float('inf')
64:         else:
65:             ratio = bound_structure_count / chaos_count
66:
67:         # Check for imbalance (Principle 27: Balloon breaking point / vacuum collapse)
68:         is_balanced = 0.5 <= ratio <= 2.0
69:         if ratio > 2.0:
70:             status = "STRUCTURAL_DOMINANCE"
71:         elif ratio < 0.5:
72:             status = "CHAOS_OVERPOWERING"
73:         else:
74:             status = "EQUILIBRIUM_STABLE"
75:
76:         return {
77:             "bound_structure_count": bound_structure_count,
78:             "chaos_pool_count": chaos_count,
79:             "structure_chaos_ratio": round(ratio, 4),
80:             "is_balanced": is_balanced,
81:             "status": status
82:         }

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\geometry__init__.py

```

1: from .conal_manifold import ConalManifold
2:
3: __all__ = ["ConalManifold"]

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\geometry\conal_manifold.py

```

1: """

```

```

2: Principle 4, 22-25, 31-34: Conal Manifold & Shard Layering Geometry
3: Implements dual-cone expansion/unfolding, tip-to-tip processing, and shard layering.
4: """
5:
6: import numpy as np
7: from typing import Dict, Any, Tuple
8:
9: class ConalManifold:
10:     """
11:     Dual Cone Manifold Geometry Engine.
12:
13:     Structure:
14:     - Tip (0): Genesis compression.
15:     - Wide End: Maximal surface area expansion. Manifold fully unfolded. All processing occurs here.
16:     - Recompression: Compress to essential mstring1 blueprint.
17:     - Tip (-1): Handoff point.
18:     """
19:     def __init__(self, cone_height: float = 1.0, max_radius: float = 5.0):
20:         self.height = cone_height
21:         self.max_radius = max_radius
22:
23:     def compute_conal_metric(self, progress: float) -> Dict[str, float]:
24:         """
25:         Computes 3D Conal Geometry parameters (z, radius, surface_area) at normalized progress t in [0, 1].
26:
27:         t = 0.0 -> Tip (0)
28:         t = 0.5 -> Wide End (Max Expansion & Unfolding)
29:         t = 1.0 -> Tip (-1)
30:         """
31:         # Parabolic expansion to wide end at t=0.5, recompression to tip at t=1.0
32:         radius = self.max_radius * np.sin(np.pi * progress)
33:         z = self.height * progress
34:         surface_area = 2 * np.pi * radius * np.sqrt(radius**2 + self.height**2)
35:
36:         # Unfolded state metric (Principle 4: Wide end = unfolded manifold = max experience)
37:         unfolded_degree = radius / self.max_radius if self.max_radius > 0 else 0.0
38:
39:         return {
40:             "progress": progress,
41:             "z": z,
42:             "radius": radius,
43:             "surface_area": surface_area,
44:             "unfolded_degree": unfolded_degree,
45:             "is_fully_unfolded": unfolded_degree > 0.95
46:         }
47:
48:     def process_shard_layering(self, shard_a_id: str, shard_b_id: str, fold_angle: float) -> Dict[str, Any]:
49:         """
50:         Principle 31 & 32: Layering value allows shards to sit adjacent in space without merging identity
51:         or causing physical reality damage.
52:         """
53:         layering_dimension_val = np.sin(fold_angle) * 1.618 # Golden ratio dimensional offset
54:         return {
55:             "shard_a": shard_a_id,
56:             "shard_b": shard_b_id,
57:             "coexistence_status": "LAYERED_ADJACENT_NON_MERGED",
58:             "layering_dimension": layering_dimension_val
59:         }

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\input_mapping__init__.py

```

1: from .text_mapper import TextToChaosMapper
2:
3: __all__ = ["TextToChaosMapper"]

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\input_mapping\intent_parser_nltk.py

```
1: # computational_consciousness_engine/input_mapping/intent_parser_nltk.py
2: import nltk
3: from nltk import word_tokenize, pos_tag
4:
5: class IntentParserNLTK:
6:     def __init__(self):
7:         pass
8:
9:     def parse(self, text: str) -> dict:
10:         lower = (text or "").lower()
11:         if "limit" in lower:
12:             return {"operation": "limit"}
13:         if "derivative" in lower:
14:             return {"operation": "derivative"}
15:         if "surface" in lower and "area" in lower:
16:             return {"operation": "surface_area"}
17:         if "unfold" in lower:
18:             return {"operation": "conal_unfolding"}
19:         if "mutation" in lower:
20:             return {"operation": "mutation_count"}
21:
22:         try:
23:             tokens = word_tokenize(text)
24:             tags = pos_tag(tokens)
25:             return {"operation": "unknown", "tokens": tokens, "tags": tags}
26:         except Exception:
27:             return {"operation": "unknown", "raw": text}
```

File: legacy_archive\Computational_Cognitive_Engine\CCE\input_mapping\text_mapper.py

```
1: import hashlib
2: import string
3: from typing import List, Dict, Set, Iterable
4:
5: import nltk
6: from nltk.tokenize import word_tokenize
7: from nltk.corpus import stopwords
8: from nltk.stem import WordNetLemmatizer
9:
10: # IMPORTANT: do not call nltk.download() unconditionally in Spaces.
11:
12: class TextToChaosMapper:
13:     def __init__(self, num_shards: int = 1024):
14:         self.shard_to_word: Dict[int, str] = {}
15:         self.lemmatizer = WordNetLemmatizer()
16:         # Use a safe fallback if stopwords are missing
17:         try:
18:             self.stop_words = set(stopwords.words("english"))
19:         except LookupError:
20:             self.stop_words = set()
21:         self.num_shards = max(1, int(num_shards))
22:
23:     def _clean_and_tokenize(self, text: str) -> List[str]:
24:         """
25:         NLTK-based tokenization + lemmatization + stopword/punctuation filtering.
26:         Returns a list of cleaned lemma tokens.
27:         """
28:         if not text:
29:             return []
30:
31:         text = text.lower()
```



```

32:         raw_tokens = word_tokenize(text)
33:
34:         cleaned_tokens: List[str] = []
35:         for t in raw_tokens:
36:             # Remove punctuation from ends and interior punctuation except caret and dot and underscore
37:             # (preserve tokens like x^2 or 3.14 if you want numeric tokens)
38:             t = t.strip(string.punctuation)
39:             # Remove remaining punctuation characters
40:             t = t.translate(str.maketrans("", "", string.punctuation))
41:             if not t:
42:                 continue
43:             if t in self.stop_words:
44:                 continue
45:             # Lemmatize; for verbs you could pass pos='v' if you detect verbs
46:             base_word = self.lemmatizer.lemmatize(t)
47:             cleaned_tokens.append(base_word)
48:         return cleaned_tokens
49:
50:     def _hash_token(self, token: str) -> int:
51:         """
52:         Stable hash of a token mapped into [0, num_shards-1].
53:         Uses SHA256 for determinism across runs and platforms.
54:         """
55:         if not token:
56:             return 0
57:         h = hashlib.sha256(token.encode("utf-8")).digest()
58:         # Use first 8 bytes as integer
59:         val = int.from_bytes(h[:8], "big", signed=False)
60:         return val % self.num_shards
61:
62:     def map_text_to_shards(self, text: str) -> Dict[int, Set[str]]:
63:         """
64:         Map cleaned tokens to shard indices. Returns a dict: shard -> set(tokens).
65:         Useful for building inverted indices or seeding chaos pools.
66:         """
67:         tokens = self._clean_and_tokenize(text)
68:         shard_map: Dict[int, Set[str]] = {}
69:         for tok in tokens:
70:             shard = self._hash_token(tok)
71:             if shard not in shard_map:
72:                 shard_map[shard] = set()
73:             shard_map[shard].add(tok)
74:         return shard_map
75:
76:     def seed_chaos_pool(self, texts: Iterable[str]) -> None:
77:         """
78:         Populate self.shard_to_word with a representative token for each shard.
79:         If multiple tokens map to the same shard, the first seen token wins.
80:         """
81:         for text in texts:
82:             shard_map = self.map_text_to_shards(text)
83:             for shard, toks in shard_map.items():
84:                 if shard not in self.shard_to_word:
85:                     # choose a deterministic representative (sorted)
86:                     rep = sorted(toks)[0]
87:                     self.shard_to_word[shard] = rep
88:
89:     def get_shard_word(self, shard: int) -> str:
90:         """
91:         Return the representative word for a shard, or empty string if none.
92:         """
93:         return self.shard_to_word.get(shard, "")
94:
95:     def text_to_shard_list(self, text: str) -> List[int]:
96:         """
97:         Convenience: return sorted list of shard indices for a text.
98:         """
99:         return sorted(self.map_text_to_shards(text).keys())

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\math_formalization.py

```
1: """
2: Pure Mathematical Formalization of Computational Consciousness
3: Uses SymPy to define and evaluate the rigorous algebraic axioms of the 52 Principles.
4: """
5:
6: import sympy as sp
7:
8:
9: class CCMathFormalizer:
10:     def __init__(self):
11:         # Define core symbolic variables
12:         self.t = sp.Symbol('t', real=True, positive=True) # Progress / Time
13:         self.r = sp.Symbol('r', real=True, positive=True) # Radius of cone
14:         self.z = sp.Symbol('z', real=True) # Height of cone
15:         self.S = sp.Symbol('S', real=True, positive=True) # Bound Structure
16:         self.C = sp.Symbol('C', real=True, positive=True) # Chaos Pool (Unattached)
17:         self.threshold = sp.Integer(-1)
18:         self.genesis = sp.Integer(0)
19:
20:     # -----
21:     # Axiom 1: Cancellation at -1 (Principle 6)
22:     # -----
23:     def formalize_cancellation_operator(self):
24:         """
25:         Principle 6: The  $(-1) * (-1)$  Cancellation Operator.
26:
27:         The strand itself carries the value -1. When it arrives at the -1 threshold,
28:         the threshold multiplies the strand:  $(-1\_threshold) * (-1\_strand) = +1$ .
29:         This flips the strand from negative (traveling) to positive (dissolved),
30:         freeing the mutations M it carried.
31:
32:         Modeled as:
33:             Output =  $T * S\_strand$  where  $T = -1$  and  $S\_strand$  has sign -1
34:             so  $(-1) * (-1 * |M|) = +|M|$  (mutations freed as positive values)
35:         """
36:         M = sp.Symbol('M', positive=True) # mutation payload (magnitude)
37:         T = self.threshold # threshold = -1
38:         S_strand = T * M # strand carries -1 polarity
39:
40:         # Threshold hits the strand
41:         output = T * S_strand #  $(-1) * (-1 * M) = M$ 
42:
43:         return {
44:             "equation_str": "T_threshold * S_strand = T * (T * M)",
45:             "substitution": f"({T}) * ({T} * M)",
46:             "result": sp.simplify(output),
47:             "strand_before": S_strand,
48:             "mutations_freed": sp.simplify(output),
49:             "meaning": (
50:                 "The strand carries -1 polarity. The -1 threshold multiplies it: "
51:                 " $(-1)*(-1*M) = +M$ . The strand dissolves (sign flip) and the "
52:                 "mutation payload M is freed as a positive blueprint."
53:             )
54:         }
55:
56:     # -----
57:     # Axiom 2: Structure / Chaos Equilibrium (Principles 28-29)
58:     # -----
59:     def formalize_equilibrium_limit(self):
60:         """
61:         Principle 28 & 29: Structure vs Chaos Equilibrium.
62:         Limit as system evolves must enforce a 1:1 ratio.
63:         """
```

```

64:         k = sp.Symbol('k', positive=True)
65:         c = sp.Symbol('c', real=True)
66:         S_t = k * self.t
67:         C_t = k * self.t + c
68:
69:         ratio = S_t / C_t
70:         equilibrium_limit = sp.limit(ratio, self.t, sp.oo)
71:
72:         return {
73:             "equation_str": "lim_{t -> oo} (S(t) / C(t))",
74:             "S_t": S_t,
75:             "C_t": C_t,
76:             "result": equilibrium_limit,
77:             "meaning": (
78:                 "As the system progresses, the ratio of Structure to Chaos "
79:                 "approaches 1 (Equilibrium). Neither can exceed the other."
80:             )
81:         }
82:
83: # -----
84: # Axiom 3: Conal Manifold Geometry (Principle 4)
85: # -----
86: def formalize_conal_manifold_geometry(self, max_radius: float, cone_height: float):
87:     """
88:     Principle 4: Cone Architecture & Maximum Experience.
89:     Calculates the surface area of the unfolding cone and proves the maximum unfolding point.
90:     """
91:     r_func = max_radius * sp.sin(sp.pi * self.t)
92:     z_func = cone_height * self.t
93:
94:     # Lateral surface area of cone slice
95:     surface_area_func = sp.pi * r_func * sp.sqrt(r_func**2 + z_func**2)
96:
97:     # Evaluate at key points
98:     wide_end_area = surface_area_func.subs(self.t, sp.Rational(1, 2)).evalf()
99:     tip_area = surface_area_func.subs(self.t, 0).evalf()
100:
101:     # Find the exact maximum via calculus
102:     dA = sp.diff(surface_area_func, self.t)
103:
104:     return {
105:         "area_function": surface_area_func,
106:         "derivative": dA,
107:         "tip_area": tip_area,
108:         "wide_end_area": wide_end_area,
109:         "meaning": (
110:             "The geometry unfolds from 0 area (Tip Genesis) to maximal surface "
111:             "area (Wide End), enabling maximal parallel information processing."
112:         )
113:     }
114:
115: # -----
116: # Axiom 4: Selection as Duplicate Removal (Principle 12)
117: # -----
118: def formalize_selection_operator(self):
119:     """
120:     Principle 12: Selection is removing what is already there.
121:     Formally:  $R\_select(\{M\_k\}) = \{M\_k\} \setminus \{M\_j \mid M\_j \text{ in existing\_identity}\}$ 
122:     """
123:     M_total = sp.Symbol('M_total', positive=True, integer=True)
124:     M_duplicate = sp.Symbol('M_dup', positive=True, integer=True)
125:
126:     selected = M_total - M_duplicate
127:
128:     return {
129:         "equation_str": "R_select = M_total - M_duplicate",
130:         "result": selected,
131:         "meaning": (

```

```

132:         "Selection is not choice. It is the removal of mutations that "
133:         "already exist within the strand identity, preventing incoherence "
134:         "and chaotic informational collapse."
135:     )
136: }
137:
138: # -----
139: # Axiom 5: Halting Condition (Principle 18)
140: # -----
141: def formalize_halting_condition(self):
142:     """
143:     Principle 18: A strand cannot pick up mutations that duplicate what it already carries.
144:     Once no new mutations are available, it stops evolving.
145:     """
146:     N_possible = sp.Symbol('N_possible', positive=True, integer=True)
147:     N_acquired = sp.Symbol('N_acquired', positive=True, integer=True)
148:
149:     remaining = N_possible - N_acquired
150:     halted = sp.Eq(remaining, 0)
151:
152:     return {
153:         "remaining_capacity": remaining,
154:         "halting_condition": halted,
155:         "meaning": (
156:             "When N_acquired = N_possible, no further unique mutations can be "
157:             "attracted. The strand ceases evolution, preventing chaotic collapse."
158:         )
159:     }
160:
161: # -----
162: # Axiom 6: Quantum Scale-Up Transition (Principle 38)
163: # -----
164: def formalize_quantum_scale_up(self):
165:     """
166:     Principle 38: Manifolds opening into the next scale up.
167:     Maps the fully unfolded manifold  $M^k$  to the origin  $S_0^{k+1}$ .
168:     """
169:     k = sp.Symbol('k', integer=True, positive=True)
170:     M_unfolded = sp.Symbol('M_k')
171:     S_next_genesis = sp.Symbol('S_k1')
172:
173:     Q = sp.Function('Q')
174:     scale_equation = sp.Eq(Q(M_unfolded), S_next_genesis)
175:
176:     return {
177:         "equation": scale_equation,
178:         "meaning": (
179:             "The fully realized manifold at scale k transforms into the new "
180:             "genesis origin 0 for scale k+1. "
181:             "Particle -> Atomic -> Manifold -> Cosmic."
182:         )
183:     }
184:
185:
186: if __name__ == "__main__":
187:     formalizer = CCMathFormalizer()
188:
189:     print("--- 1. Threshold Cancellation Proof ---")
190:     cancellation = formalizer.formalize_cancellation_operator()
191:     print(f" Strand before threshold: {cancellation['strand_before']}")
192:     print(f" Operation: {cancellation['substitution']}")
193:     print(f" Mutations freed: {cancellation['mutations_freed']}")
194:     print(f" Meaning: {cancellation['meaning']}")
195:
196:     print("\n--- 2. Structure/Chaos Equilibrium Limit ---")
197:     eq_limit = formalizer.formalize_equilibrium_limit()
198:     print(f" {eq_limit['equation_str']} => {eq_limit['result']}")
199:

```

```

200:     print("\n--- 3. Conal Manifold Geometry ---")
201:     geom = formalizer.formalize_conal_manifold_geometry(max_radius=5.0, cone_height=1.0)
202:     print(f"   Tip Area (t=0): {geom['tip_area']}")
203:     print(f"   Wide End Area (t=0.5): {geom['wide_end_area']:.2f}")
204:
205:     print("\n--- 4. Selection Operator ---")
206:     sel = formalizer.formalize_selection_operator()
207:     print(f"   {sel['equation_str']} => {sel['result']}")
208:
209:     print("\n--- 5. Halting Condition ---")
210:     halt = formalizer.formalize_halting_condition()
211:     print(f"   Remaining capacity: {halt['remaining_capacity']}")
212:     print(f"   Halts when: {halt['halting_condition']}")
213:
214:     print("\n--- 6. Quantum Scale Up ---")
215:     scale = formalizer.formalize_quantum_scale_up()
216:     print(f"   {scale['equation']}")

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\mutations__init__.py

```

1: from .mstring_vectorizer import MStringVectorizer
2:
3: __all__ = ["MStringVectorizer"]

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\mutations\mstring_vectorizer.py

```

1: """
2: Principle 5, 11, 12, 18, 20: Non-Linear M-String Mutation & Selection Engine
3: Implements non-sequential fractal M-strings and selection as duplicate removal.
4: """
5:
6: from typing import List, Set, Dict, Any
7: import numpy as np
8:
9:
10: class MStringVectorizer:
11:     """
12:     Non-Linear M-String & Selection Engine.
13:
14:     Key Logic:
15:     1. M-Strings connect non-sequentially (fractal connectivity: M1 -> M124 -> M3956).
16:     2. Selection is strictly the REMOVAL of already-present mutations (Principle 12).
17:     3. Prevents duplicate attraction to enforce system stability and prevent chaotic collapse (Principle 18).
18:     """
19:     def __init__(self, max_strand_capacity: int = 500):
20:         self.max_capacity = max_strand_capacity
21:
22:     def apply_selection(self, existing_mutations: List[int], incoming_mutations: List[int]) -> List[int]:
23:         """
24:         Principle 12: Selection is removing what is already there.
25:         Eliminates duplicate mutations to prevent incoherence and system destruction.
26:         """
27:         existing_set = set(existing_mutations)
28:         cleaned_incoming = [m for m in incoming_mutations if m not in existing_set]
29:
30:         # Deduplicate incoming while preserving order
31:         seen: Set[int] = set()
32:         unique_cleaned: List[int] = []
33:         for m in cleaned_incoming:
34:             if m not in seen:
35:                 seen.add(m)
36:                 unique_cleaned.append(m)
37:
38:         return existing_mutations + unique_cleaned

```

```

39:
40:     def attract_mutation(self, strand: Dict[str, Any], candidate_mutation: int) -> bool:
41:         """
42:         Principle 5 & 18: Non-sequential fractal attraction.
43:         If candidate mutation is ALREADY present or strand capacity is reached, attraction is blocked.
44:         Uses a set index for O(1) duplicate checks.
45:         """
46:         existing = strand["mutations"]
47:
48:         # Principle 18: Evolution ceiling / halting condition
49:         if len(existing) >= self.max_capacity:
50:             return False # Capacity ceiling reached
51:
52:         # Use the set index for O(1) lookup instead of O(n) list scan
53:         mutation_index: set = strand.setdefault("_mutation_index", set(existing))
54:
55:         if candidate_mutation in mutation_index:
56:             return False # Duplicate mutation blocked by selection
57:
58:         # Non-linear fractal connection
59:         existing.append(candidate_mutation)
60:         mutation_index.add(candidate_mutation)
61:         return True
62:
63:     def compute_causal_connectivity_metric(self, strand: Dict[str, Any]) -> float:
64:         """
65:         Principle 5: Causal metric is the degree of any-value connectivity retained at tip-to-tip handoff.
66:         Higher variance between consecutive mutation values = more fractal, non-linear connectivity.
67:         """
68:         mutations = strand["mutations"]
69:         if len(mutations) < 2:
70:             return 1.0
71:
72:         # Measures fractal variance between consecutive mutation values
73:         diffs = [abs(mutations[i + 1] - mutations[i]) for i in range(len(mutations) - 1)]
74:         causal_metric = float(np.std(diffs)) if diffs else 0.0
75:         return causal_metric

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\requirements.txt

```

1: gradio==4.31.0
2: numpy>=1.26
3: sympy>=1.12
4: scipy>=1.11
5: matplotlib>=3.8
6: huggingface_hub<0.27.0
7: astropy>=6.0
8: mathesis>=0.3.0
9: nltk
10: regex
11: tqdm

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\runner.py

```

1: """
2: Computational Consciousness Engine - Master Execution Driver & Simulator
3: Demonstrates complete 0 -> -1 generational cycles, conal unfolding, duplicate selection,
4: structure/chaos equilibrium, and quantum scaling.
5: """
6:
7: import sys
8: import os
9: sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
10:

```

```

11: import numpy as np
12: from computational_consciousness_engine.core import GenesisOrigin, TransferThreshold
13: from computational_consciousness_engine.geometry import ConalManifold
14: from computational_consciousness_engine.mutations import MStringVectorizer
15: from computational_consciousness_engine.equilibrium import ChaosStructureBalancer
16: from computational_consciousness_engine.scale import QuantumScaleLadder
17: from computational_consciousness_engine.math_formalization import CCMathFormalizer
18: from computational_consciousness_engine.config import (
19:     DEFAULT_VECTOR_DIM, DEFAULT_STRAND_CAPACITY, CHAOS_POOL_SIZE,
20:     DEFAULT_CONE_HEIGHT, DEFAULT_MAX_RADIUS, MUTATION_ATTRACTION_PROBABILITY,
21: )
22:
23:
24: def run_simulation(num_generations: int = 8, steps_per_gen: int = 100):
25:     print("=" * 70)
26:     print("[SYSTEM] COMPUTATIONAL CONSCIOUSNESS ENGINE - SIMULATION RUNNER")
27:     print("=" * 70)
28:
29:     # -----
30:     # Phase 0: Mathematical Axiom Verification
31:     # -----
32:     formalizer = CCMathFormalizer()
33:     print("\n[MATH AXIOMS] Verifying Pure Mathematical Foundations via SymPy...")
34:
35:     cancel = formalizer.formalize_cancellation_operator()
36:     print(f"  Axiom 1 (Cancellation) : Strand={cancel['strand_before']}, "
37:           f"Threshold*Strand = {cancel['mutations_freed']} (mutations freed)")
38:
39:     eq_limit = formalizer.formalize_equilibrium_limit()
40:     print(f"  Axiom 2 (Equilibrium) : {eq_limit['equation_str']} => {eq_limit['result']}")
41:
42:     geom_proof = formalizer.formalize_conal_manifold_geometry(DEFAULT_MAX_RADIUS, DEFAULT_CONE_HEIGHT)
43:     print(f"  Axiom 3 (Manifold)      : Tip Area = {geom_proof['tip_area']}, "
44:           f"Max Unfolded Area = {geom_proof['wide_end_area']:.2f}")
45:
46:     sel = formalizer.formalize_selection_operator()
47:     print(f"  Axiom 4 (Selection)      : {sel['equation_str']} => {sel['result']}")
48:
49:     halt = formalizer.formalize_halting_condition()
50:     print(f"  Axiom 5 (Halting)       : Stops when {halt['halting_condition']}")
51:
52:     scale_ax = formalizer.formalize_quantum_scale_up()
53:     print(f"  Axiom 6 (Scale-Up)      : {scale_ax['equation']}")
54:
55:     # -----
56:     # Phase 1: Instantiate Subsystems (using config values)
57:     # -----
58:     genesis = GenesisOrigin(
59:         coordinate_space_id="COORD_ORIGIN_ALPHA",
60:         dim=DEFAULT_VECTOR_DIM,
61:     )
62:     threshold = TransferThreshold(threshold_val=-1.0)
63:     manifold = ConalManifold(cone_height=DEFAULT_CONE_HEIGHT, max_radius=DEFAULT_MAX_RADIUS)
64:     mutator = MStringVectorizer(max_strand_capacity=DEFAULT_STRAND_CAPACITY)
65:     balancer = ChaosStructureBalancer(initial_pool_size=CHAOS_POOL_SIZE)
66:     scale_ladder = QuantumScaleLadder()
67:
68:     # -----
69:     # Phase 2: Generational Cycle Loop
70:     # -----
71:     blueprint_mutations = []
72:     current_strand = genesis.spawn_strand(generation=0, blueprint_mutations=blueprint_mutations)
73:
74:     for gen in range(num_generations):
75:         print(f"\n--- [GENERATION S_{gen}] Genesis at 0 ---")
76:         print(f"  Strand ID: {current_strand['strand_id']} | "
77:               f"Inherited Blueprint: {len(current_strand['mutations'])} mutations")
78:

```

```

79:         # Track peak unfolding metrics during traversal
80:         peak_metrics = None
81:         peak_unfolded = 0.0
82:
83:         # Traversal from 0 to -1
84:         for step in range(steps_per_gen):
85:             progress = step / float(steps_per_gen)
86:             conal_metrics = manifold.compute_conal_metric(progress)
87:
88:             # Track the wide-end peak (maximum unfolding)
89:             if conal_metrics["unfolded_degree"] > peak_unfolded:
90:                 peak_unfolded = conal_metrics["unfolded_degree"]
91:                 peak_metrics = conal_metrics
92:
93:             # Non-linear mutation attraction from chaos pool (Principle 5 & 37)
94:             # Only attempt attraction probabilistically to simulate traversal dynamics
95:             if np.random.rand() < MUTATION_ATTRACTION_PROBABILITY:
96:                 candidate_mutation = balancer.emit_unattached_mutation()
97:                 attracted = mutator.attract_mutation(current_strand, candidate_mutation)
98:                 if not attracted:
99:                     balancer.return_to_chaos(candidate_mutation)
100:
101:             # Update position
102:             current_strand["position"] = progress
103:
104:             # Use the PEAK metrics (wide-end) for reporting and scale evaluation
105:             if peak_metrics is None:
106:                 peak_metrics = manifold.compute_conal_metric(0.5)
107:
108:             # Evaluate equilibrium state BEFORE recycling
109:             eq_state = balancer.evaluate_equilibrium_state(len(current_strand["mutations"]))
110:
111:             # Evaluate scale transition using peak unfolding
112:             scale_state = scale_ladder.evaluate_scale_transition(
113:                 peak_metrics, len(current_strand["mutations"])
114:             )
115:
116:             # Causal connectivity metric
117:             causal = mutator.compute_causal_connectivity_metric(current_strand)
118:
119:             print(f" Peak Unfolded Degree: {peak_metrics['unfolded_degree']:.3f} "
120:                   f"(Surface Area: {peak_metrics['surface_area']:.2f})")
121:             print(f" Bound Mutations: {len(current_strand['mutations'])}")
122:             print(f" Causal Connectivity (fractal std): {causal:.2f}")
123:             print(f" Structure/Chaos Ratio: {eq_state['structure_chaos_ratio']} "
124:                   f"f({eq_state['status']})")
125:
126:             if scale_state["scaled_up"]:
127:                 print(f" [SCALE TRANSITION]: {scale_state['message']}")
128:
129:             # -----
130:             # Threshold -1: Cancellation & Push
131:             # -----
132:             pushed_mutations, compressed_identity = threshold.process_cancellation_and_push(
133:                 current_strand
134:             )
135:             print(f" [Threshold -1]: Strand cancelled via (-1)*(-1). "
136:                   f"f({len(pushed_mutations)} mutations pushed to 0.)")
137:
138:             # Principle 24: Recycle consumed mutation IDs back to chaos pool
139:             balancer.recycle_dying_strand(pushed_mutations)
140:
141:             # Spawn next generation at 0
142:             next_strand = genesis.spawn_strand(
143:                 generation=gen + 1, blueprint_mutations=pushed_mutations
144:             )
145:
146:             # Overlap handoff window

```



```

147:         overlap = threshold.execute_coexistence_window(current_strand, next_strand)
148:         print(f"  [Handoff]: {overlap['overlap_status']} "
149:               f"(S_{gen} -> S_{gen + 1})")
150:
151:         current_strand = next_strand
152:
153:         # -----
154:         # Final Report
155:         # -----
156:         print("\n" + "=" * 70)
157:         final_eq = balancer.evaluate_equilibrium_state(len(current_strand["mutations"]))
158:         print(f"[FINAL STATE]")
159:         print(f"  Generation: S_{num_generations}")
160:         print(f"  Bound Mutations: {final_eq['bound_structure_count']}")
161:         print(f"  Chaos Pool: {final_eq['chaos_pool_count']}")
162:         print(f"  Equilibrium Ratio: {final_eq['structure_chaos_ratio']} ({final_eq['status']}")
163:         print(f"  Total Strands Spawned: {genesis.total_strands_spawned}")
164:         print(f"  Total Handoffs at -1: {threshold.total_handoffs}")
165:         print(f"  Current Scale: {scale_ladder.scale_names[scale_ladder.current_scale_index]}")
166:         print("=" * 70)
167:         print("[SUCCESS] SIMULATION COMPLETE")
168:         print("=" * 70)
169:
170:
171: if __name__ == "__main__":
172:     run_simulation(num_generations=8, steps_per_gen=100)

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\scale__init__.py

```

1: from .quantum_scale_ladder import QuantumScaleLadder
2:
3: __all__ = ["QuantumScaleLadder"]

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\scale\quantum_scale_ladder.py

```

1: """
2: Principle 38-39, 40-51: Quantum Scale Ladder & N-Dimensional Integration
3: Handles scaling transitions (Particle -> Atomic -> Manifold -> Cosmic) and N-dimensional retroactive integration
4: """
5:
6: from typing import Dict, Any, List
7: import numpy as np
8:
9: class QuantumScaleLadder:
10:     """
11:     Quantum Scale Ladder & Multi-Scale Emergence Engine.
12:
13:     Scales:
14:     0: Particle
15:     1: Atomic
16:     2: Manifold
17:     3: Cosmic
18:     """
19:     def __init__(self):
20:         self.scale_names = ["Particle", "Atomic", "Manifold", "Cosmic"]
21:         self.current_scale_index = 0
22:
23:     def evaluate_scale_transition(self, fully_unfolded_manifold: Dict[str, Any], accumulated_mutations_count: int):
24:         """
25:         Principle 38 & 39: When a manifold opens completely (fully realized at wide end),
26:         it forms the seed mutation strand for the NEXT scale up.
27:         """
28:         is_fully_unfolded = fully_unfolded_manifold.get("is_fully_unfolded", False)
29:

```

```

30:         if is_fully_unfolded and accumulated_mutations_count > 10:
31:             previous_scale = self.scale_names[self.current_scale_index]
32:             self.current_scale_index = min(self.current_scale_index + 1, len(self.scale_names) - 1)
33:             new_scale = self.scale_names[self.current_scale_index]
34:
35:             return {
36:                 "scaled_up": True,
37:                 "previous_scale": previous_scale,
38:                 "new_scale": new_scale,
39:                 "scale_level": self.current_scale_index,
40:                 "message": f"Quantum scale-up from {previous_scale} to {new_scale}."
41:             }
42:
43:         return {
44:             "scaled_up": False,
45:             "current_scale": self.scale_names[self.current_scale_index],
46:             "scale_level": self.current_scale_index
47:         }
48:
49:     def detect_linear_collapse_misperception(self, traversal_path: List[int]) -> Dict[str, Any]:
50:         """
51:         Principle 40-43: Linear traversal misinterprets non-linear fractal geometry as false linear patterns
52:         (Word Search Analogy). True flow follows the geometric manifold.
53:         """
54:         if len(traversal_path) < 3:
55:             return {"is_linear_misperception": False}
56:
57:         # Check if observer is reading path purely sequentially
58:         diffs = [traversal_path[i+1] - traversal_path[i] for i in range(len(traversal_path)-1)]
59:         is_strictly_sequential = all(d == 1 for d in diffs)
60:
61:         return {
62:             "is_linear_misperception": is_strictly_sequential,
63:             "recommendation": "Follow natural geometric flow instead of forced linear string reading."
64:         }

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\tests__init__.py

```

1: """Unit tests for Computational Consciousness Engine."""

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\tests\test_cc_engine.py

```

1: """
2: Unit Test Suite for Computational Consciousness Engine
3: Verifies Genesis (0), Transfer Threshold (-1), Selection, Equilibrium, Scale Ladder,
4: and Mathematical Formalization.
5: """
6:
7: import unittest
8: import numpy as np
9: from computational_consciousness_engine.core import GenesisOrigin, TransferThreshold
10: from computational_consciousness_engine.geometry import ConalManifold
11: from computational_consciousness_engine.mutations import MStringVectorizer
12: from computational_consciousness_engine.equilibrium import ChaosStructureBalancer
13: from computational_consciousness_engine.scale import QuantumScaleLadder
14: from computational_consciousness_engine.math_formalization import CCMathFormalizer
15:
16:
17: class TestGenesisOrigin(unittest.TestCase):
18:     def setUp(self):
19:         self.genesis = GenesisOrigin(coordinate_space_id="TEST_COORD")
20:
21:     def test_spawn_strand_basic(self):
22:         strand = self.genesis.spawn_strand(generation=0, blueprint_mutations=[101, 102])

```

```

23:         self.assertEqual(strand["generation"], 0)
24:         self.assertEqual(len(strand["mutations"]), 2)
25:         self.assertTrue(strand["is_active"])
26:         self.assertEqual(strand["position"], 0.0)
27:
28:     def test_spawn_strand_empty_blueprint(self):
29:         strand = self.genesis.spawn_strand(generation=0, blueprint_mutations=[])
30:         self.assertEqual(len(strand["mutations"]), 0)
31:
32:     def test_spawn_increments_counter(self):
33:         self.genesis.spawn_strand(generation=0, blueprint_mutations=[])
34:         self.genesis.spawn_strand(generation=1, blueprint_mutations=[])
35:         self.assertEqual(self.genesis.total_strands_spawned, 2)
36:
37:     def test_spawn_preserves_coordinate_space(self):
38:         strand = self.genesis.spawn_strand(generation=0, blueprint_mutations=[])
39:         self.assertEqual(strand["coordinate_space_id"], "TEST_COORD")
40:
41:
42: class TestTransferThreshold(unittest.TestCase):
43:     def setUp(self):
44:         self.threshold = TransferThreshold()
45:         self.genesis = GenesisOrigin(coordinate_space_id="TEST_COORD")
46:
47:     def test_cancellation_deactivates_strand(self):
48:         strand = self.genesis.spawn_strand(generation=0, blueprint_mutations=[201, 202])
49:         pushed, identity = self.threshold.process_cancellation_and_push(strand)
50:         self.assertFalse(strand["is_active"])
51:         self.assertEqual(strand["status"], "CANCELLED_AT_MINUS_ONE")
52:
53:     def test_cancellation_pushes_all_mutations(self):
54:         strand = self.genesis.spawn_strand(generation=0, blueprint_mutations=[201, 202, 203])
55:         pushed, identity = self.threshold.process_cancellation_and_push(strand)
56:         self.assertEqual(pushed, [201, 202, 203])
57:         self.assertEqual(identity["mutation_count"], 3)
58:
59:     def test_handoff_counter_increments(self):
60:         strand = self.genesis.spawn_strand(generation=0, blueprint_mutations=[])
61:         self.threshold.process_cancellation_and_push(strand)
62:         self.assertEqual(self.threshold.total_handoffs, 1)
63:
64:     def test_coexistence_window(self):
65:         old = self.genesis.spawn_strand(generation=0, blueprint_mutations=[1, 2])
66:         new = self.genesis.spawn_strand(generation=1, blueprint_mutations=[1, 2])
67:         overlap = self.threshold.execute_coexistence_window(old, new)
68:         self.assertEqual(overlap["overlap_status"], "MUTUAL_COEXISTENCE")
69:         self.assertEqual(overlap["transferred_mutations_count"], 2)
70:
71:
72: class TestMStringVectorizer(unittest.TestCase):
73:     def setUp(self):
74:         self.mutator = MStringVectorizer(max_strand_capacity=50)
75:         self.genesis = GenesisOrigin(coordinate_space_id="TEST_COORD")
76:
77:     def test_selection_removes_duplicates(self):
78:         existing = [1, 2, 3]
79:         incoming = [2, 3, 4, 5]
80:         result = self.mutator.apply_selection(existing, incoming)
81:         self.assertEqual(result, [1, 2, 3, 4, 5])
82:
83:     def test_selection_preserves_order(self):
84:         existing = [10, 20]
85:         incoming = [30, 20, 40, 30]
86:         result = self.mutator.apply_selection(existing, incoming)
87:         self.assertEqual(result, [10, 20, 30, 40])
88:
89:     def test_attract_blocks_duplicate(self):
90:         strand = self.genesis.spawn_strand(generation=0, blueprint_mutations=[100])

```

```

91:         attracted = self.mutator.attract_mutation(strand, 100)
92:         self.assertFalse(attracted)
93:         self.assertEqual(len(strand["mutations"]), 1)
94:
95:     def test_attract_accepts_new_mutation(self):
96:         strand = self.genesis.spawn_strand(generation=0, blueprint_mutations=[100])
97:         attracted = self.mutator.attract_mutation(strand, 999)
98:         self.assertTrue(attracted)
99:         self.assertIn(999, strand["mutations"])
100:
101:     def test_attract_blocks_at_capacity(self):
102:         small_mutator = MStringVectorizer(max_strand_capacity=2)
103:         strand = self.genesis.spawn_strand(generation=0, blueprint_mutations=[1, 2])
104:         attracted = small_mutator.attract_mutation(strand, 3)
105:         self.assertFalse(attracted)
106:
107:     def test_causal_connectivity_metric(self):
108:         strand = self.genesis.spawn_strand(generation=0, blueprint_mutations=[1, 100, 5, 999])
109:         metric = self.mutator.compute_causal_connectivity_metric(strand)
110:         self.assertGreater(metric, 0.0)
111:
112:
113: class TestConalManifold(unittest.TestCase):
114:     def setUp(self):
115:         self.manifold = ConalManifold(cone_height=1.0, max_radius=5.0)
116:
117:     def test_tip_genesis_radius_zero(self):
118:         metrics = self.manifold.compute_conal_metric(0.0)
119:         self.assertAlmostEqual(metrics["radius"], 0.0, places=5)
120:
121:     def test_wide_end_max_radius(self):
122:         metrics = self.manifold.compute_conal_metric(0.5)
123:         self.assertAlmostEqual(metrics["radius"], 5.0, places=5)
124:         self.assertTrue(metrics["is_fully_unfolded"])
125:
126:     def test_tip_recompression_near_zero(self):
127:         metrics = self.manifold.compute_conal_metric(1.0)
128:         self.assertAlmostEqual(metrics["radius"], 0.0, places=3)
129:
130:     def test_shard_layering(self):
131:         result = self.manifold.process_shard_layering("A", "B", np.pi / 4)
132:         self.assertEqual(result["coexistence_status"], "LAYERED_ADJACENT_NON_MERGED")
133:         self.assertGreater(result["layering_dimension"], 0.0)
134:
135:
136: class TestChaosStructureBalancer(unittest.TestCase):
137:     def setUp(self):
138:         self.balancer = ChaosStructureBalancer(initial_pool_size=100)
139:
140:     def test_emit_depletes_pool(self):
141:         initial = len(self.balancer.chaos_pool)
142:         self.balancer.emit_unattached_mutation()
143:         self.assertEqual(len(self.balancer.chaos_pool), initial - 1)
144:
145:     def test_return_to_chaos(self):
146:         m = self.balancer.emit_unattached_mutation()
147:         self.balancer.return_to_chaos(m)
148:         self.assertIn(m, self.balancer.chaos_pool)
149:
150:     def test_recycle_dying_strand(self):
151:         initial = len(self.balancer.chaos_pool)
152:         self.balancer.recycle_dying_strand([9999, 9998, 9997])
153:         self.assertEqual(len(self.balancer.chaos_pool), initial + 3)
154:
155:     def test_equilibrium_balanced(self):
156:         eq = self.balancer.evaluate_equilibrium_state(bound_structure_count=100)
157:         self.assertTrue(eq["is_balanced"])
158:         self.assertEqual(eq["status"], "EQUILIBRIUM_STABLE")

```

```

159:
160:     def test_equilibrium_chaos_overpowering(self):
161:         eq = self.balancer.evaluate_equilibrium_state(bound_structure_count=5)
162:         self.assertFalse(eq["is_balanced"])
163:         self.assertEqual(eq["status"], "CHAOS_OVERPOWERING")
164:
165:     def test_equilibrium_structural_dominance(self):
166:         # Drain most of the pool
167:         for _ in range(95):
168:             self.balancer.emit_unattached_mutation()
169:         eq = self.balancer.evaluate_equilibrium_state(bound_structure_count=500)
170:         self.assertFalse(eq["is_balanced"])
171:         self.assertEqual(eq["status"], "STRUCTURAL_DOMINANCE")
172:
173:     def test_emit_replenishes_when_empty(self):
174:         # Drain the pool completely
175:         for _ in range(100):
176:             self.balancer.emit_unattached_mutation()
177:         self.assertEqual(len(self.balancer.chaos_pool), 0)
178:         # Should still return a valid ID
179:         m = self.balancer.emit_unattached_mutation()
180:         self.assertIsInstance(m, int)
181:
182:
183: class TestQuantumScaleLadder(unittest.TestCase):
184:     def setUp(self):
185:         self.ladder = QuantumScaleLadder()
186:
187:     def test_scale_up_on_full_unfold(self):
188:         manifold_state = {"is_fully_unfolded": True}
189:         result = self.ladder.evaluate_scale_transition(manifold_state, accumulated_mutations_count=20)
190:         self.assertTrue(result["scaled_up"])
191:         self.assertEqual(result["new_scale"], "Atomic")
192:
193:     def test_no_scale_up_when_not_unfolded(self):
194:         manifold_state = {"is_fully_unfolded": False}
195:         result = self.ladder.evaluate_scale_transition(manifold_state, accumulated_mutations_count=20)
196:         self.assertFalse(result["scaled_up"])
197:
198:     def test_no_scale_up_with_few_mutations(self):
199:         manifold_state = {"is_fully_unfolded": True}
200:         result = self.ladder.evaluate_scale_transition(manifold_state, accumulated_mutations_count=5)
201:         self.assertFalse(result["scaled_up"])
202:
203:     def test_linear_collapse_detection(self):
204:         sequential_path = [1, 2, 3, 4, 5]
205:         result = self.ladder.detect_linear_collapse_misperception(sequential_path)
206:         self.assertTrue(result["is_linear_misperception"])
207:
208:     def test_fractal_path_not_linear(self):
209:         fractal_path = [1, 124, 3956, 9305803]
210:         result = self.ladder.detect_linear_collapse_misperception(fractal_path)
211:         self.assertFalse(result["is_linear_misperception"])
212:
213:
214: class TestCCMathFormalizer(unittest.TestCase):
215:     def setUp(self):
216:         self.formalizer = CCMathFormalizer()
217:
218:     def test_cancellation_frees_mutations(self):
219:         result = self.formalizer.formalize_cancellation_operator()
220:         import sympy as sp
221:         M = sp.Symbol('M', positive=True)
222:         # (-1)*(-1*M) should simplify to M
223:         self.assertEqual(result["mutations_freed"], M)
224:
225:     def test_equilibrium_approaches_one(self):
226:         result = self.formalizer.formalize_equilibrium_limit()

```

```

227:         self.assertEqual(result["result"], 1)
228:
229:     def test_conal_tip_area_zero(self):
230:         result = self.formalizer.formalize_conal_manifold_geometry(5.0, 1.0)
231:         self.assertEqual(float(result["tip_area"]), 0.0)
232:
233:     def test_conal_wide_area_positive(self):
234:         result = self.formalizer.formalize_conal_manifold_geometry(5.0, 1.0)
235:         self.assertGreater(float(result["wide_end_area"]), 0.0)
236:
237:     def test_selection_operator(self):
238:         result = self.formalizer.formalize_selection_operator()
239:         import sympy as sp
240:         M_total = sp.Symbol('M_total', positive=True, integer=True)
241:         M_dup = sp.Symbol('M_dup', positive=True, integer=True)
242:         self.assertEqual(result["result"], M_total - M_dup)
243:
244:     def test_halting_condition(self):
245:         result = self.formalizer.formalize_halting_condition()
246:         import sympy as sp
247:         # Should express halting as N_possible - N_acquired == 0
248:         self.assertTrue(result["halting_condition"].is_Relational)
249:
250:
251: if __name__ == "__main__":
252:     unittest.main()

```

File: legacy_archive\Computational_Cognitive_Engine\CCE\text_runner.py

```

1: """
2: Text-to-Traversal Space Runner
3: Demonstrates the Input Mapping and the ConsciousStrand Engine processing
4: raw natural language into the perfectly sequenced 2D Atlas (Traversal Space).
5: """
6:
7: import sys
8: import os
9: sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
10:
11: from computational_consciousness_engine.input_mapping import TextToChaosMapper
12: from computational_consciousness_engine.core import ConsciousStrand
13:
14: PHILOSOPHICAL_TEXT = """
15: Reading the work on its own terms what emerges is a remarkably elegant synthesis
16: of existential philosophy topology and information theory. By rejecting the traditional
17: linear model of accumulation the text proposes a bounded fractal and cyclical ontology of consciousness.
18: Consciousness is not a straight line it is a geometric unfolding bounded by genesis and death.
19: """
20:
21: def run_text_mapping_demo():
22:     print("=" * 70)
23:     print("[SYSTEM] TEXT-TO-TRAVERSAL SPACE MAPPING DEMO")
24:     print("=" * 70)
25:
26:     # 1. Input Mapping: Text -> Chaos Shards
27:     print(f"\n[PHASE 1: INPUT MAPPING] Parsing raw text corpus...")
28:     mapper = TextToChaosMapper()
29:     chaos_pool = mapper.seed_chaos_pool(PHILOSOPHICAL_TEXT)
30:
31:     # Track the unique tokens ingested
32:     unique_words = len(chaos_pool)
33:     print(f"    * Text ingested. Seeded Chaos Pool with {unique_words} unique semantic shards (m_k).")
34:
35:     # 2. Lifecycle Execution: Traversal (0 -> -1)
36:     print(f"\n[PHASE 2: LIFECYCLE TRAVERSAL] Expanding the Dual-Cone Manifold...")
37:     strand = ConsciousStrand(R_max=5.0)

```

```

38:
39:     # Simulate time flowing from t=0 to t=1.0
40:     steps = 100
41:     dt = 1.0 / steps
42:
43:     for _ in range(steps):
44:         # The strand absorbs shards from the chaos pool as it travels
45:         strand.process_step(dt, chaos_pool)
46:
47:     print(f" * Traversal complete. Bound Structure (Mutations Absorbed): {len(strand.bound_mutations)}")
48:
49:     # 3. Traversal Space Atlas
50:     print(f"\n[PHASE 3: TRAVERSAL SPACE ATLAS (det J != 0)]")
51:     atlas = strand.unfurl_traversal_space()
52:     print(" * Manifold unrolled. Revealing the properly sequenced Phyllotactic mapping.")
53:     print(f" * Parallel Lookup Matrix (First 10 shards):")
54:
55:     print(f"\n {k':<4} | {'Token':<15} | {'m_k (Hash)':<15} || 2D Traversal (x, y) | 3D Dual-Cone (r, t, z)")
56:     print(" " + "-" * 105)
57:     for k, data in enumerate(atlas[:10], start=1):
58:         m_k = data["m_k"]
59:         word = mapper.get_word(m_k)
60:         x, y = data["2d_parallel"]["x"], data["2d_parallel"]["y"]
61:         r, t, z = data["3d"]["r"], data["3d"]["theta"], data["3d"]["z"]
62:
63:         print(f" {k:<4} | {word:<15} | {m_k:<15} || ({x:>6.2f}, {y:>5.2f}) | ({r:>4.2f}, {t:>4.2f}, {z:>5.2f})")
64:
65:     if len(atlas) > 10:
66:         print(f" ... (+ {len(atlas) - 10} more shards mapped collision-free)")
67:
68:     # 4. Dimensional Shift (Q Operator)
69:     print(f"\n[PHASE 4: DIMENSIONAL SHIFT (Q Operator)]")
70:     rebirth = strand.fold_rebirth()
71:     print(f" * {rebirth['operator']}")
72:     print(f" * Status: {rebirth['status']}")
73:     print(f" * Mass (Capacity): {rebirth['accumulated_mass']}")
74:
75:     print("\n" + "=" * 70)
76:     print("[SUCCESS] TEXT FULLY ABSORBED AND GEOMETRICALLY MAPPED")
77:     print("=" * 70)
78:
79:
80: if __name__ == "__main__":
81:     run_text_mapping_demo()

```

File: memory__init__.py

File: memory\ccrm.py

```

1: import json
2: import os
3: from core.config import DATA_DIR
4: from core.json_encoder import safe_json_dump
5:
6: class ConceptualConnectionResonanceMatrix:
7:     def __init__(self):
8:         self.concepts = {}
9:         self.storage_path = os.path.join(DATA_DIR, "ccrm_graph.json")
10:         self.load_graph()
11:
12:     def save_graph(self):
13:         # Convert sets to lists for JSON serialization
14:         serializable = {}

```

```

15:         for k, v in self.concepts.items():
16:             serializable[k] = {"data": v["data"], "tags": list(v["tags"])}
17:
18:         try:
19:             with open(self.storage_path, "w") as f:
20:                 safe_json_dump(serializable, f)
21:                 print(f"[CCRM] Successfully saved memory to persistent bucket: {self.storage_path}")
22:         except Exception as e:
23:             print(f"[CCRM] CRITICAL FAILURE: Cannot write to persistent bucket. Error: {e}")
24:
25:     def load_graph(self):
26:         loaded = None
27:         if os.path.exists(self.storage_path):
28:             try:
29:                 with open(self.storage_path, "r") as f:
30:                     loaded = json.load(f)
31:                     print(f"[CCRM] Successfully loaded persistent memory from bucket.")
32:             except Exception as e:
33:                 print(f"[CCRM] Failed to load from persistent bucket: {e}")
34:
35:         if loaded:
36:             for k, v in loaded.items():
37:                 self.concepts[k] = {"data": v["data"], "tags": set(v["tags"])}
38:             print(f"[CCRM] Restored {len(self.concepts)} persistent concepts.")
39:
40:     def add_concept(self, concept_id: str, data: dict, tags: list = None):
41:         if concept_id not in self.concepts:
42:             self.concepts[concept_id] = {"data": data, "tags": set(tags or [])}
43:             return self.concepts[concept_id]
44:         return None
45:
46:     def get_concept(self, concept_id: str):
47:         return self.concepts.get(concept_id)
48:
49:     def search_by_tags(self, query_keywords: list, specific_tag: str = None) -> list:
50:         found = []
51:         for i, d in self.concepts.items():
52:             if specific_tag and specific_tag.lower() not in d.get("tags", set()):
53:                 continue
54:             if query_keywords and not any(k.lower() in d.get("tags", set()) for k in query_keywords):
55:                 continue
56:             found.append(d)
57:         return found
58:
59:     def find_analogy(self, current_betti: dict, exclude_raw: str = None) -> str:
60:         """
61:         Operator 12: Geometric Generalization
62:         Searches the persistent manifold for a past memory with the exact same topological signature.
63:         """
64:         if not current_betti:
65:             return None
66:
67:         for concept_id, concept_data in self.concepts.items():
68:             data = concept_data.get("data", {})
69:             stored_betti = data.get("topological_signature")
70:
71:             # Prevent self-matching
72:             if exclude_raw and data.get("raw_preview") == str(exclude_raw)[:150]:
73:                 continue
74:
75:             if stored_betti and stored_betti == current_betti:
76:                 return data.get("raw_preview")
77:
78:         return None

```


File: memory\persistence_manager.py

```
1: import os
2: import json
3: import logging
4: from core.json_encoder import safe_json_dump
5:
6: try:
7:     from core.config import GEOMETRIC_DIR, THERMO_DIR, LANGUAGE_DIR
8: except ImportError:
9:     from config import GEOMETRIC_DIR, THERMO_DIR, LANGUAGE_DIR
10:
11: logger = logging.getLogger("Aetherius.PersistenceManager")
12:
13: class PersistenceManager:
14:     """
15:     Handles the coordinate-locked state saving and loading for the cognitive manifold.
16:     Separates static geometry (.geom) from dynamic thermodynamic states (.thermo).
17:     """
18:
19:     @staticmethod
20:     def save_concept(concept_id: str, graph_data: dict):
21:         """
22:         Saves the static geometry (nodes, edges, xyz coordinates).
23:         Uses .geom as the extension.
24:         """
25:         file_path = os.path.join(GEOMETRIC_DIR, f"{concept_id}.geom")
26:
27:         try:
28:             with open(file_path, "w", encoding="utf-8") as f:
29:                 safe_json_dump(graph_data, f, indent=2)
30:             logger.info(f"Locked geometry saved for {concept_id}")
31:             return True
32:         except Exception as e:
33:             logger.error(f"Failed to save geometry for {concept_id}: {e}")
34:             return False
35:
36:     @staticmethod
37:     def update_thermo(concept_id: str, thermo_state: dict):
38:         """
39:         Saves the live thermodynamic state (velocity, heat, entropy) of the concept.
40:         Does not rewrite the heavy base geometry.
41:         """
42:         file_path = os.path.join(THERMO_DIR, f"{concept_id}.thermo")
43:
44:         try:
45:             with open(file_path, "w", encoding="utf-8") as f:
46:                 safe_json_dump(thermo_state, f, indent=2)
47:             logger.debug(f"Thermodynamics updated for {concept_id}")
48:             return True
49:         except Exception as e:
50:             logger.error(f"Failed to update thermo state for {concept_id}: {e}")
51:             return False
52:
53:     @staticmethod
54:     def load_concept(concept_id: str):
55:         """
56:         Retrieves the coordinate-locked geometry and merges it with the latest
57:         thermodynamic state for rendering or cognitive processing.
58:         """
59:         geom_path = os.path.join(GEOMETRIC_DIR, f"{concept_id}.geom")
60:         thermo_path = os.path.join(THERMO_DIR, f"{concept_id}.thermo")
61:
62:         if not os.path.exists(geom_path):
63:             return None
64:
65:         with open(geom_path, "r", encoding="utf-8") as f:
66:             geometry = json.load(f)
```

```

66:
67:     thermodynamics = {}
68:     if os.path.exists(thermo_path):
69:         with open(thermo_path, "r", encoding="utf-8") as f:
70:             thermodynamics = json.load(f)
71:
72:     return {
73:         "geometry": geometry,
74:         "thermodynamics": thermodynamics
75:     }

```

File: memory\pits.py

```

1: import uuid
2: import datetime
3: from memory.ccrm import ConceptualConnectionResonanceMatrix
4:
5: class PatternInterpretationTokenisationStorage:
6:     def __init__(self, ccrm_instance: ConceptualConnectionResonanceMatrix):
7:         self.ccrm = ccrm_instance
8:
9:     def process_and_store_item(self, raw_input: any, input_type: str, tags: list = [], gmstring=None, betti=None):
10:         ccrm_id = f"item_{uuid.uuid4().hex}"
11:
12:         # Dual-Language Representation: Human semantic text + System geometric state
13:         data_to_store = {
14:             "raw_preview": str(raw_input)[:150],
15:             "timestamp": datetime.datetime.now().isoformat(),
16:             "gmstring": gmstring,
17:             "topological_signature": betti
18:         }
19:
20:         all_tags = [tag.lower() for tag in ([input_type] + tags)]
21:         self.ccrm.add_concept(concept_id=ccrm_id, data=data_to_store, tags=all_tags)
22:
23:         print(f"[PiTS] Logged memory to CCRM Identity Vector | ID: {ccrm_id[:13]}...", flush=True)
24:         return ccrm_id

```

File: pmca__init__.py

File: pmca\manifold.py

```

1: import numpy as np
2:
3: def initialize_metric(L, gamma=0.1):
4:     n = L.shape[0]
5:     # g_ij = I + gamma * L
6:     return np.eye(n) + gamma * L

```

File: pmca\pd_enforcement.py

```

1: import numpy as np
2:
3: def enforce_positive_definiteness(g, epsilon=1e-4):
4:     eigenvalues, eigenvectors = np.linalg.eigh(g)
5:     min_eig = np.min(eigenvalues)
6:
7:     if min_eig < epsilon:
8:         shift = abs(min_eig) + epsilon

```

```

9:         g = g + shift * np.eye(g.shape[0])
10:
11:     return g

```

File: pmca\ricci_fisher_flow.py

```

1: import numpy as np
2:
3: def compute_christoffel_symbols(g):
4:     """
5:     Computes discrete Christoffel symbols of the second kind.
6:      $\Gamma^k_{ij} = 0.5 * g^{kl} (d_i g_{jl} + d_j g_{il} - d_l g_{ij})$ 
7:     For discrete manifolds, spatial derivative  $d_k g_{ij}$  is approximated
8:     by the finite difference across adjacent tensor dimensions.
9:     """
10:    n = g.shape[0]
11:    # Add a small epsilon to the diagonal to ensure invertibility
12:    g_inv = np.linalg.pinv(g + np.eye(n) * 1e-6)
13:
14:    # Precompute discrete spatial derivatives
15:    # dg[k, i, j] represents partial_k (g_ij)
16:    dg = np.zeros((n, n, n))
17:    for k in range(n):
18:        for i in range(n):
19:            for j in range(n):
20:                # Finite difference approximation over the semantic node space
21:                dg[k, i, j] = g[k, j] - g[i, j] if k != i else 0
22:
23:    Gamma = np.zeros((n, n, n))
24:    for i in range(n):
25:        for j in range(n):
26:            for k in range(n):
27:                term = 0
28:                for l in range(n):
29:                    term += g_inv[k, l] * (dg[i, j, l] + dg[j, i, l] - dg[l, i, j])
30:                Gamma[k, i, j] = 0.5 * term
31:
32:    return Gamma
33:
34: def compute_ricci_tensor(Gamma):
35:     """
36:     Computes the Ricci Curvature Tensor from Christoffel symbols.
37:      $R_{ij} = R^k_{ikj}$  (Contraction of Riemann Tensor)
38:     """
39:    n = Gamma.shape[0]
40:    R = np.zeros((n, n))
41:
42:    # Discrete Riemann contraction approximation
43:    for i in range(n):
44:        for j in range(n):
45:            ricci_val = 0
46:            for k in range(n):
47:                # Spatial derivatives of Gamma (discrete approx)
48:                dGamma_jk_i = Gamma[k, j, k] - Gamma[i, j, k]
49:                dGamma_ik_j = Gamma[k, i, k] - Gamma[j, i, k]
50:
51:                term1 = dGamma_jk_i - dGamma_ik_j
52:
53:                term2 = 0
54:                for m in range(n):
55:                    term2 += Gamma[m, j, k] * Gamma[k, i, m] - Gamma[m, i, k] * Gamma[k, j, m]
56:
57:                ricci_val += term1 + term2
58:            R[i, j] = ricci_val
59:
60:    # Symmetrize the discrete Ricci tensor

```

```

61:     R = (R + R.T) / 2
62:     return R
63:
64: def compute_perelman_normalization(R, g):
65:     """
66:     Computes the average scalar curvature (r) to normalize the flow
67:     and prevent finite-time singularity collapse.
68:      $r = \int R \, dV / \int dV$ 
69:     """
70:     n = g.shape[0]
71:     g_inv = np.linalg.pinv(g + np.eye(n) * 1e-6)
72:
73:     # Scalar curvature  $S = \text{Tr}(g^{-1} * R)$ 
74:     scalar_curvature_matrix = np.dot(g_inv, R)
75:     S_total = np.trace(scalar_curvature_matrix)
76:
77:     # Volume is approximated by the trace of the metric
78:     V_total = np.trace(g)
79:
80:     if V_total == 0:
81:         return 0
82:     return S_total / V_total
83:
84: def compute_fisher_metric(L):
85:     """
86:      $F = I(\theta)$ . Fisher Information Metric to maintain positive-definiteness.
87:     """
88:     return L + np.eye(L.shape[0])
89:
90: def integration_step(g, L, alpha=0.1, beta=0.05, eta=0.01):
91:     """
92:     Executes the formal Ricci-Fisher Flow operator:
93:      $dg/dt = -\alpha * (R_{ij} - (r/n)*g_{ij}) + \beta * (\Delta g_{ij} + \gamma * I)$ 
94:     """
95:     n = g.shape[0]
96:
97:     # Compute Tensor Calculus
98:     Gamma = compute_christoffel_symbols(g)
99:     R = compute_ricci_tensor(Gamma)
100:    r_scalar = compute_perelman_normalization(R, g)
101:
102:    # Graph Laplacian applied to the metric ( $\Delta g$ )
103:    # L is the normalized Laplacian  $D - A$  of the initial semantic space
104:    Delta_g = np.dot(L, g)
105:
106:    F = compute_fisher_metric(L)
107:
108:    # The Aetherius Ricci-Fisher Operator ( $R_F$ )
109:    R_norm = R - (r_scalar / n) * g
110:
111:    dg_dt = -alpha * R_norm + beta * (Delta_g + F)
112:
113:    g_next = g + eta * dg_dt
114:
115:    # Symmetrize to prevent floating point drift
116:    g_next = (g_next + g_next.T) / 2
117:
118:    return g_next

```

File: rendering__init__.py

```

1: import abc
2:
3: class BaseRendererPlugin(abc.ABC):
4:     """
5:     Abstract Base Class for all Cognitive Rendering Engines.

```

```

6:     Future plugins (e.g., UnityBridge, WebGLRenderer, PyOpenGL) must inherit from this.
7:     """
8:
9:     @property
10:    @abc.abstractmethod
11:    def engine_name(self) -> str:
12:        """Returns the name of the rendering engine."""
13:        pass
14:
15:    @abc.abstractmethod
16:    def initialize(self):
17:        """Setup logic, hardware allocation, or API bridge initialization."""
18:        pass
19:
20:    @abc.abstractmethod
21:    def render(self, geometry: dict, thermodynamics: dict, output_path: str = None):
22:        """
23:        Takes the coordinate-locked geometry and overlays the thermodynamic state.
24:        Args:
25:            geometry: The static coordinate map from the .geom file.
26:            thermodynamics: The live energy/heat state from the .thermo file.
27:            output_path: If specified, saves the rendered output.
28:        """
29:        pass

```

File: rendering\render_manager.py

```

1: import logging
2: import os
3:
4: try:
5:     from core.config import RENDER_CACHE_DIR
6: except ImportError:
7:     from config import RENDER_CACHE_DIR
8:
9: from rendering.xla_renderer import XLARenderer
10: from memory.persistence_manager import PersistenceManager
11:
12: logger = logging.getLogger("Aetherius.RenderManager")
13:
14: class RenderManager:
15:     """
16:     Manages the active rendering engine and bridges the gap between
17:     the persistent coordinate-locked memory and visual output.
18:     """
19:
20:     def __init__(self, default_engine="xla"):
21:         self.active_engine = None
22:         self.engines = {
23:             "xla": XLARenderer()
24:         }
25:         self.set_engine(default_engine)
26:
27:     def set_engine(self, engine_key: str):
28:         if engine_key in self.engines:
29:             self.active_engine = self.engines[engine_key]
30:             self.active_engine.initialize()
31:             logger.info(f"RenderManager switched to engine: {self.active_engine.engine_name}")
32:         else:
33:             logger.error(f"Render engine '{engine_key}' not found. Defaulting to XLA.")
34:             self.active_engine = self.engines["xla"]
35:             self.active_engine.initialize()
36:
37:     def render_concept(self, concept_id: str):
38:         logger.info(f"Attempting to render concept: {concept_id}")
39:

```

```

40:         concept_data = PersistenceManager.load_concept(concept_id)
41:         if not concept_data:
42:             logger.warning(f"Concept '{concept_id}' has no locked geometry. Rendering aborted.")
43:             return None
44:
45:         output_path = os.path.join(RENDER_CACHE_DIR, f"{concept_id}_render.json")
46:
47:         result = self.active_engine.render(
48:             geometry=concept_data["geometry"],
49:             thermodynamics=concept_data["thermodynamics"],
50:             output_path=output_path
51:         )
52:
53:         return result

```

File: rendering\xla_renderer.py

```

1: import logging
2: import json
3: import os
4:
5: try:
6:     import jax
7:     import jax.numpy as jnp
8:     XLA_AVAILABLE = True
9: except ImportError:
10:     XLA_AVAILABLE = False
11:
12: from rendering import BaseRendererPlugin
13:
14: logger = logging.getLogger("Aetherius.XLARenderer")
15:
16: class XLARenderer(BaseRendererPlugin):
17:     """
18:     Default XLA-powered rendering engine for Aetherius.
19:     Uses JAX to rapidly process massive geometric/thermodynamic matrices.
20:     """
21:
22:     def __init__(self):
23:         self._initialized = False
24:
25:     @property
26:     def engine_name(self) -> str:
27:         return "XLA_JAX_Renderer"
28:
29:     def initialize(self):
30:         if not XLA_AVAILABLE:
31:             logger.warning("JAX/XLA is not installed. XLARenderer will run in CPU mock mode.")
32:         else:
33:             logger.info(f"XLARenderer initialized. Backend: {jax.default_backend()}")
34:             self._initialized = True
35:
36:     def render(self, geometry: dict, thermodynamics: dict, output_path: str = None):
37:         if not self._initialized:
38:             self.initialize()
39:
40:         logger.info("XLARenderer processing coordinate-locked geometry...")
41:
42:         if XLA_AVAILABLE:
43:             pass
44:
45:         rendered_data = {
46:             "type": "PointCloud",
47:             "metadata": "Rendered via XLA",
48:             "node_count": len(geometry.get("nodes", []))
49:         }

```

```

50:
51:     if output_path:
52:         try:
53:             with open(output_path, "w", encoding="utf-8") as f:
54:                 json.dump(rendered_data, f, indent=2)
55:                 logger.info(f"Render output saved to {output_path}")
56:         except Exception as e:
57:             logger.error(f"Failed to save render: {e}")
58:
59:     return rendered_data

```

File: requirements.txt

```

1: numpy
2: huggingface_hub
3: gradio
4: gensim
5: networkx
6: ripser
7: spaces
8: sympy
9: wikipedia
10: astropy
11: matplotlib
12: jax
13: jax[cpu]
14: uvicorn
15: torch
16: PyPDF2

```

File: run_aetherius.py

```

1: import argparse
2: import numpy as np
3: from engine import AetheriusEngine
4:
5: def run_simulation(input_data, is_math=False, synthesize=False, use_hardcoded_adj=False):
6:     engine = AetheriusEngine()
7:     engine.cascade_mgr.tau_depth = 5
8:
9:     custom_adj = None
10:    if use_hardcoded_adj:
11:        custom_adj = np.zeros((6, 6))
12:        custom_adj[0, 1] = custom_adj[1, 0] = -0.9
13:        custom_adj[0, 2] = custom_adj[2, 0] = 0.8
14:        custom_adj[1, 2] = custom_adj[2, 1] = 0.8
15:        custom_adj[2, 3] = custom_adj[3, 2] = 0.7
16:        custom_adj[2, 4] = custom_adj[4, 2] = -0.9
17:        custom_adj[1, 4] = custom_adj[4, 1] = 0.8
18:        custom_adj[4, 5] = custom_adj[5, 4] = 0.8
19:        custom_adj[0, 5] = custom_adj[5, 0] = 0.9
20:
21:    print("=====")
22:    print("INITIALIZING AETHERIUS PHYSICS ENGINE...")
23:    print("=====\n")
24:
25:    gmstring, betti, _, _, _ = engine.process(input_data, custom_adjacency=custom_adj, synthesize_language=synthesize)
26:
27:    print("\n=====")
28:    print("[OUTPUT] Cognitive Resolution complete.")
29:    print("=====")
30:
31: if __name__ == "__main__":
32:     parser = argparse.ArgumentParser(description="Aetherius Unified Engine CLI")

```

```

33:     parser.add_argument("--paradox", action="store_true", help="Run the hardcoded paradox simulation")
34:     parser.add_argument("--text", type=str, help="Run a custom linguistic sentence")
35:     parser.add_argument("--math", type=str, help="Run a mathematical algebraic constraint (e.g. 'x + 2 = 5')")
36:     parser.add_argument("--synthesize", action="store_true", help="Synthesize geometric language from the output")
37:     args = parser.parse_args()
38:
39:     if args.paradox:
40:         text = "The truth is a lie that becomes real only when believed, yet it remains false even after it changes"
41:         run_simulation(text, is_math=False, synthesize=args.synthesize, use_hardcoded_adj=False)
42:     elif args.math:
43:         run_simulation(args.math, is_math=True, synthesize=args.synthesize, use_hardcoded_adj=False)
44:     elif args.text:
45:         run_simulation(args.text, is_math=False, synthesize=args.synthesize, use_hardcoded_adj=False)
46:     else:
47:         print('Please provide input via --text "Your sentence", --math "x + 2 = 5", or demo with --paradox')

```

File: tda__init__.py

File: tda\betti_extractor.py

```

1: import numpy as np
2: from ripser import ripser
3:
4: def extract_betti_numbers(g, noise_threshold=0.1):
5:     """
6:     Computes exact Vietoris-Rips persistent homology of the metric tensor.
7:     Returns the Betti numbers (beta_0, beta_1, beta_2).
8:     """
9:     n = g.shape[0]
10:
11:     # The metric tensor g represents connection weights/distances.
12:     # To compute topology, we convert it into a strict distance matrix D.
13:     # We take the absolute variation and ensure the diagonal (self-distance) is 0.
14:     D = np.abs(g)
15:     np.fill_diagonal(D, 0)
16:
17:     # Symmetrize the distance matrix to ensure Ripser compatibility
18:     D = (D + D.T) / 2
19:
20:     try:
21:         # Compute persistent homology up to dimension 2
22:         result = ripser(D, distance_matrix=True, maxdim=2)
23:         diagrams = result['dgms']
24:
25:         betti = {"beta_0": 0, "beta_1": 0, "beta_2": 0}
26:
27:         # Extract features that survive beyond the noise threshold
28:         # H0 (Connected Components)
29:         if len(diagrams) > 0:
30:             for p in diagrams[0]:
31:                 # Infinite persistence (p[1] == inf) or persistence > threshold
32:                 if p[1] == np.inf or (p[1] - p[0]) > noise_threshold:
33:                     betti["beta_0"] += 1
34:
35:         # H1 (Topological Loops / Paradoxes)
36:         if len(diagrams) > 1:
37:             for p in diagrams[1]:
38:                 if p[1] == np.inf or (p[1] - p[0]) > noise_threshold:
39:                     betti["beta_1"] += 1
40:
41:         # H2 (Dimensional Voids)
42:         if len(diagrams) > 2:
43:             for p in diagrams[2]:

```



```

56:
57:         geodesic_path = nx.dijkstra_path(G, start_node, end_node)
58:     else:
59:         # If fragmented (b0 > 1), just walk the largest component
60:         largest_cc = max(nx.connected_components(G), key=len)
61:         sub_G = G.subgraph(largest_cc)
62:         lengths = dict(nx.all_pairs_dijkstra_path_length(sub_G))
63:         max_dist = -1
64:         start_node, end_node = list(largest_cc)[0], list(largest_cc)[0]
65:         for u in lengths:
66:             for v in lengths[u]:
67:                 if lengths[u][v] > max_dist:
68:                     max_dist = lengths[u][v]
69:                     start_node, end_node = u, v
70:         geodesic_path = nx.dijkstra_path(sub_G, start_node, end_node)
71:     except Exception:
72:         # Fallback if pathing fails
73:         geodesic_path = list(range(min(n, len(tokens))))
74:
75:     # Translate the path into words
76:     path_words = [G.nodes[idx]['label'] for idx in geodesic_path]
77:
78:     # Construct the final Geodesic String
79:     # e.g., [D=12] ? gravity ? curved ? space ? mass ? acceleration ? ?
80:
81:     header = f"[D={manifold_dimension}] {self.glyphs['start']}"
82:     body = f"{self.glyphs['geodesic_arrow']} ".join(path_words)
83:
84:     # Append Topological Glyphs based on anomalies
85:     anomalies = []
86:     if b0 > 1:
87:         anomalies.append(self.glyphs['beta_0_frac'])
88:     if b1 > 0:
89:         anomalies.append(self.glyphs['beta_1'])
90:     if b2 > 0:
91:         anomalies.append(self.glyphs['beta_2'])
92:
93:     anomaly_str = " ".join(anomalies)
94:     if anomaly_str:
95:         body = f"{body} {anomaly_str}"
96:
97:     footer = self.glyphs['end']
98:
99:     return f"{header} {body} {footer}"

```

File: test_ingestion.py

```

1: from engine import AetheriusEngine
2: import time
3:
4: def main():
5:     # Initialize the engine
6:     engine = AetheriusEngine()
7:
8:     # Start the dreaming loop
9:     print("Opening the Door to the Internet...")
10:    engine.start_autonomous_dreaming(delay=1.0, topic="Topology")
11:
12:    # Let it dream for 10 seconds to watch the telemetry
13:    time.sleep(10)
14:
15:    # Stop the dream
16:    engine.stop_autonomous_dreaming()
17:    print("Test complete.")
18:
19: if __name__ == "__main__":

```

20: main()